Machine Learning Lab (23CSE301) – Exercise Solutions

Chapters 5 – 12 · Python / scikit-learn

In [1]:
```
# ---- Common imports used throughout the notebook ----
import os, re, urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import metrics

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)

# ---- Dataset loader ----
DATA_DIR = "data"
def load_data(filename, url, **read_csv_kwargs):
    """Read a real dataset. Uses ./data/<filename> if present (e.g. downl
    otherwise downloads the public copy given in `url` and caches it in .
    path = os.path.join(DATA_DIR, filename)
    if not os.path.exists(path):
        os.makedirs(DATA_DIR, exist_ok=True)
        req = urllib.request.Request(url, headers={"User-Agent": "Mozilla
        with urllib.request.urlopen(req, timeout=60) as resp, open(path,
            f.write(resp.read())
    return pd.read_csv(path, **read_csv_kwargs)

print("Libraries loaded")
```

```
Libraries loaded
```

5.8 Linear Regression – Exercises

Exercise 1 – Predict the fare of a Chicago taxi ride

Goal: predict `fare` from trip features (multiple linear regression).

> **Dataset – Chicago Taxi Trips (City of Chicago open data, 2022 extract used in Google's ML Crash Course).** Official source: https://data.cityofchicago.org/Transportation/Taxi-Trips/wrvz-psew (no Kaggle copy found). **Records:** 31,694 trips · **Features:** `trip_miles`, `trip_seconds`, `rush_hour` (derived from the pickup hour: 7–9 h or 16–18 h) · **Target:** `fare` (USD). **Preprocessing:** columns renamed; `rush_hour` flag created from `TRIP_START_HOUR`; no nulls, no zero-mile / zero-second / zero-fare trips, so no rows were removed. **Why it fits:** real metered fares are essentially *base fare + rate per mile + rate per minute*, which is exactly the linear structure the exercise asks us to model.

In [2]:
```
# 1. Import data – real Chicago taxi trips
taxi_raw = load_data("chicago_taxi_train.csv",
                     "https://raw.githubusercontent.com/Amana-Batool/Line

taxi = pd.DataFrame({
    "trip_miles":   taxi_raw["TRIP_MILES"],
    "trip_seconds": taxi_raw["TRIP_SECONDS"],
    "pickup_hour":  taxi_raw["TRIP_START_HOUR"].astype(int),
    "fare":         taxi_raw["FARE"],
})
taxi["rush_hour"] = (((taxi.pickup_hour >= 7) & (taxi.pickup_hour <= 9))
                     ((taxi.pickup_hour >= 16) & (taxi.pickup_hour <= 18)
taxi = taxi[["trip_miles", "trip_seconds", "pickup_hour", "rush_hour", "f
print(taxi.shape)
taxi.head()
```

```
(31694, 5)
```

Out[2]:

	trip_miles	trip_seconds	pickup_hour	rush_hour	fare
0	2.57	2341	7	1	31.99
1	1.18	1074	17	1	9.75
2	1.29	1173	17	1	10.25
3	3.70	3360	18	1	23.75
4	1.15	1044	17	1	10.00

In [3]:
```
# 2. Explore + check nulls
print("Null values:\n", taxi.isnull().sum(), "\n")
taxi.describe().round(2)
```

```
Null values:
 trip_miles      0
trip_seconds    0
pickup_hour     0
rush_hour       0
fare            0
dtype: int64
```

Out[3]:

	trip_miles	trip_seconds	pickup_hour	rush_hour	fare
count	31694.00	31694.00	31694.00	31694.00	31694.00
mean	8.29	1319.80	13.81	0.36	23.91
std	7.27	928.93	5.16	0.48	16.97
min	0.50	60.00	0.00	0.00	3.25
25%	1.72	548.00	10.00	0.00	9.00
50%	5.92	1081.00	14.00	0.00	18.75
75%	14.50	1888.00	18.00	1.00	38.75
max	68.12	7140.00	23.00	1.00	159.25

In [4]:
```
# 3. Features / target, split, fit
features = ["trip_miles", "trip_seconds", "rush_hour"]
X = taxi[features]
y = taxi["fare"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Intercept (b0):", round(model.intercept_, 3))
for f, c in zip(features, model.coef_):
```

```
    print(f"  {f:<13} coefficient = {c:.4f}")
```

```
Intercept (b0): 3.957
  trip_miles    coefficient = 2.0059
  trip_seconds  coefficient = 0.0026
  rush_hour     coefficient = -0.3314
```

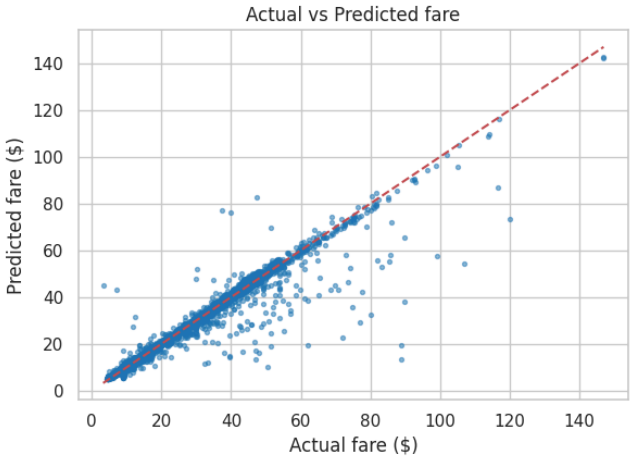
In [5]:
```
# 4. Evaluate
mse  = metrics.mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = metrics.r2_score(y_test, y_pred)
n_, p_ = X_test.shape
adj_r2 = 1 - (1 - r2) * (n_ - 1) / (n_ - p_ - 1)

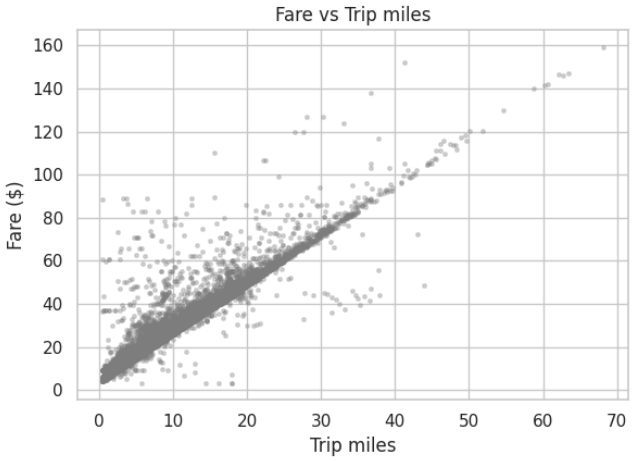
print(f"MSE         : {mse:.4f}")
print(f"RMSE        : {rmse:.4f}  (average error in $)")
print(f"R2 score    : {r2:.4f}")
print(f"Adjusted R2 : {adj_r2:.4f}")

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
ax[0].scatter(y_test, y_pred, s=8, alpha=0.5, color="tab:blue")
lims = [y_test.min(), y_test.max()]
ax[0].plot(lims, lims, "r--")
ax[0].set(title="Actual vs Predicted fare", xlabel="Actual fare ($)", yla

ax[1].scatter(taxi["trip_miles"], taxi["fare"], s=6, alpha=0.3, color="gr
ax[1].set(title="Fare vs Trip miles", xlabel="Trip miles", ylabel="Fare (
plt.tight_layout(); plt.show()
```

```
MSE         : 12.9790
RMSE        : 3.6026  (average error in $)
R2 score    : 0.9561
Adjusted R2 : 0.9561
```





In [6]:
```
# 5. Predict a new ride: 6 miles, 22 minutes, during rush hour
new_ride = pd.DataFrame([[6.0, 22 * 60, 1]], columns=features)
print(f"Predicted fare for the new ride: ${model.predict(new_ride)[0]:.2f
```

```
Predicted fare for the new ride: $19.09
```

Exercise 2 – Predict body weight from height, age and gender

Dependent variable: **Weight**; independent variables: **Height, Age, Gender** (Gender is categorical → encoded as a 0/1 dummy variable).

> **Dataset – Gym Members Exercise Dataset (Kaggle: `valakhorasani/gym-members-exercise-dataset`).** Link: https://www.kaggle.com/datasets/valakhorasani/gym-members-exercise-dataset **Records:** 973 gym members · **Features:** `Height` (m), `Age`, `Gender` · **Target:** `Weight` (kg). **Preprocessing:** kept the four relevant columns and renamed `Weight (kg)` / `Height (m)`; no nulls; Gender one-hot encoded (`Gender_Male`); 80/20 train-test split. **Why it fits:** it is a real table with exactly the variables the exercise names (weight, height, age, gender), and it is large enough for a proper train/test evaluation.

In [7]:
```
gym = load_data("gym_members_exercise_tracking.csv",
                "https://raw.githubusercontent.com/Mohammadprsd/GYM-membe
body = gym.rename(columns={"Weight (kg)": "Weight", "Height (m)": "Height
print(body.shape, "| nulls:", body.isnull().sum().sum())
body.head()
```

```
(973, 4) | nulls: 0
```

Out[7]:

	Weight	Height	Age	Gender
0	88.3	1.71	56	Male
1	74.9	1.53	46	Female
2	68.1	1.66	32	Female
3	53.2	1.70	25	Male
4	46.1	1.79	38	Male

In [8]:
```
# Encode Gender (Male = 1, Female = 0), separate X / y and split
X = pd.get_dummies(body[["Height", "Age", "Gender"]], columns=["Gender"],
y = body["Weight"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

from sklearn.linear_model import LinearRegression
lr = LinearRegression().fit(X_train, y_train)

print("Feature columns:", list(X.columns))
print(f"\nIntercept (b0): {lr.intercept_:.3f}")
```

```
for name, c in zip(X.columns, lr.coef_):
    print(f"  {name:<12}: {c:8.3f}")

print("\nEquation:  Weight = {:.2f} + {:.2f}*Height + {:.3f}*Age + {:.2f}
```

```
Feature columns: ['Height', 'Age', 'Gender_Male']

Intercept (b0): 51.179
  Height      :    8.642
  Age         :   -0.122
  Gender_Male :   22.933

Equation:  Weight = 51.18 + 8.64*Height + -0.122*Age + 22.93*Gender_Male
```

In [9]:

```
# Evaluation on the held-out test set + 5-fold cross-validation
from sklearn.model_selection import cross_val_score

pred_test = lr.predict(X_test)
r2  = metrics.r2_score(y_test, pred_test)
adj = 1 - (1 - r2) * (len(y_test) - 1) / (len(y_test) - X.shape[1] - 1)
print(f"Test R2 = {r2:.3f} | Adjusted R2 = {adj:.3f} | RMSE = {np.sqrt(me

cv_r2 = cross_val_score(LinearRegression(), X, y, cv=5, scoring="r2")
print(f"5-fold CV R2 = {cv_r2.mean():.3f} (+/- {cv_r2.std():.3f})")

plt.figure(figsize=(5.5, 5))
plt.scatter(y_test, pred_test, s=14, alpha=0.6, color="tab:blue")
lims = [y.min(), y.max()]
plt.plot(lims, lims, "r--")
plt.xlabel("Actual weight (kg)"); plt.ylabel("Predicted weight (kg)")
plt.title("Body-weight regression (test set)"); plt.show()

# Predict a new person: 1.75 m, 30 years, male
new_person = pd.DataFrame([[1.75, 30, 1.0]], columns=X.columns)
print(f"\nPredicted weight for a 1.75 m, 30-year-old male: {lr.predict(ne
```

```
Test R2 = 0.347 | Adjusted R2 = 0.336 | RMSE = 17.45 kg
5-fold CV R2 = 0.337 (+/- 0.012)
```




```
Predicted weight for a 1.75 m, 30-year-old male: 85.6 kg
```

6.6 Logistic Regression – Exercises

Exercise 1 – Graduate-school admission (GRE, GPA, institution prestige)

Binary response: `admit` (1 = likely admitted, 0 = not).

> **Dataset – Graduate Admissions (Kaggle: `mohansacharya/graduate-admissions`, file `Admission_Predict_Ver1.1.csv`).**
> Link: https://www.kaggle.com/datasets/mohansacharya/graduate-admissions It is *inspired by* the UCLA admissions data (it is not the original `admit/gre/gpa/rank` file). **Records:** 500 applicants · **Features:** `gre` (GRE score, /340), `gpa` (CGPA, /10), `rank` (University Rating 1–5; **5 = most prestigious**, the opposite

> direction of the UCLA `rank`) · **Target:** `admit`. **Preprocessing:** the dataset has no yes/no label – only `Chance of Admit` (0.34–0.97) – so `admit = 1` when `Chance of Admit >= 0.75` (≈ 44 % of applicants). `rank` is one-hot encoded. TOEFL, SOP, LOR and Research are not used because the exercise only names GRE, GPA and prestige. **Why it fits:** same three predictors as the exercise (test score, GPA, institution prestige) on real applicant data.

In [10…
```
grad_raw = load_data("Admission_Predict_Ver1.1.csv",
                     "https://raw.githubusercontent.com/divyansha1115/Gra
grad_raw.columns = grad_raw.columns.str.strip()          # 'LOR ' / 'Chan

adm = pd.DataFrame({
    "admit": (grad_raw["Chance of Admit"] >= 0.75).astype(int),
    "gre":   grad_raw["GRE Score"],
    "gpa":   grad_raw["CGPA"],
    "rank":  grad_raw["University Rating"],
})
print(adm.shape)
print("Null values:", adm.isnull().sum().sum())
print("\nAdmission rate:", adm["admit"].mean().round(3))
adm.head()
```

```
(500, 4)
Null values: 0

Admission rate: 0.436
```

Out[10]:

	admit	gre	gpa	rank
0	1	337	9.65	4
1	1	324	8.87	4
2	0	316	8.00	3
3	1	322	8.67	3
4	0	314	8.21	2

In [11…
```
# Prestige (rank) is categorical -> one-hot encode; GRE / GPA are numeric
X = pd.concat([adm[["gre", "gpa"]], pd.get_dummies(adm["rank"], prefix="r
y = adm["admit"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,

from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(C=1e6, max_iter=5000)      # C very large ~ no r
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]
```

```
coef = pd.DataFrame({"feature": X.columns, "coefficient": clf.coef_[0], "
print("Intercept:", clf.intercept_.round(3))
coef.round(4)
```

```
Intercept: [-68.049]
```

Out[11]:

	feature	coefficient	odds_ratio
0	gre	0.0872	1.0911
1	gpa	4.5941	98.8979
2	rank_2	-0.5919	0.5533
3	rank_3	0.5572	1.7457
4	rank_4	0.7505	2.1180
5	rank_5	1.7723	5.8844

In [12…

```
print("Confusion matrix:\n", metrics.confusion_matrix(y_test, y_pred))
print(f"\nAccuracy : {metrics.accuracy_score(y_test, y_pred):.3f}")
print(f"ROC-AUC  : {metrics.roc_auc_score(y_test, y_prob):.3f}\n")
print(metrics.classification_report(y_test, y_pred, target_names=["Not ad

fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
sns.heatmap(pd.crosstab(y_test, y_pred, rownames=["Actual"], colnames=["P
ax[0].set_title("Confusion matrix — admission")
fpr, tpr, _ = metrics.roc_curve(y_test, y_prob)
ax[1].plot(fpr, tpr, label=f"AUC = {metrics.auc(fpr, tpr):.2f}"); ax[1].p
ax[1].set(title="ROC curve", xlabel="False Positive Rate", ylabel="True P
plt.tight_layout(); plt.show()
```

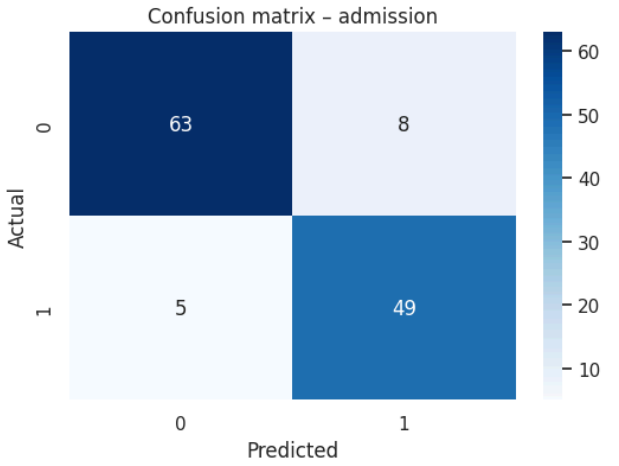
```
Confusion matrix:
 [[63  8]
 [ 5 49]]

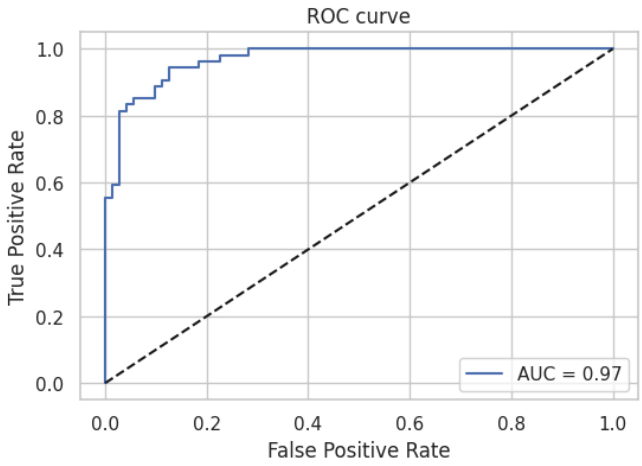
Accuracy : 0.896
ROC-AUC  : 0.968

              precision    recall  f1-score   support

Not admitted       0.93      0.89      0.91        71
    Admitted       0.86      0.91      0.88        54

    accuracy                           0.90       125
   macro avg       0.89      0.90      0.89       125
weighted avg       0.90      0.90      0.90       125
```





In [13…
```python
# Admission probability for a new applicant: GRE 320, CGPA 9.0 - top-rate
def applicant(gre, gpa, rating):
    row = {"gre": gre, "gpa": gpa, **{f"rank_{r}": int(rating == r) for r
    return pd.DataFrame([row], columns=X.columns)

for rating in (5, 1):
    print(f"P(admit) for GRE=320, CGPA=9.0, University Rating={rating} :
```

```
P(admit) for GRE=320, CGPA=9.0, University Rating=5 : 95.07%
P(admit) for GRE=320, CGPA=9.0, University Rating=1 : 76.64%
```

Exercise 2 – Is a person susceptible to a certain disease?

> **Dataset – Pima Indians Diabetes Database (Kaggle: `uciml/pima-indians-diabetes-database`).** Link: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database **Records:** 768 women · **Features:** `Pregnancies`, `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, `BMI`, `DiabetesPedigreeFunction`, `Age` · **Target:** `disease` (= `Outcome`, 1 = tested positive for diabetes). **Preprocessing:** in `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin` and `BMI` a value of 0 is physiologically impossible, so 0 is treated as *missing* and replaced by the training-set median; features are standardised. **Why it fits:** a real medical table with a binary "susceptible / not susceptible" outcome – the classic logistic-regression medical example (replaces the DATAtab *Medical example* file, which is not on Kaggle).

In [14…
```python
pima = load_data("diabetes.csv",
                 "https://raw.githubusercontent.com/plotly/datasets/maste
```

```
med = pima.rename(columns={"Outcome": "disease"})

# 0 is not a valid measurement for these columns -> treat as missing
zero_is_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin"
med[zero_is_missing] = med[zero_is_missing].replace(0, np.nan)

print(med.shape, "| disease prevalence:", med.disease.mean().round(3))
print("Missing values per column (after 0 -> NaN):", med.isnull().sum()[m
med.head()
```

```
(768, 9) | disease prevalence: 0.349
Missing values per column (after 0 -> NaN): {'Glucose': 5, 'BloodPressur
e': 35, 'SkinThickness': 227, 'Insulin': 374, 'BMI': 11}
```

Out[14]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148.0	72.0	35.0	NaN	33.6	0
1	1	85.0	66.0	29.0	NaN	26.6	0
2	8	183.0	64.0	NaN	NaN	23.3	0
3	1	89.0	66.0	23.0	94.0	28.1	0
4	0	137.0	40.0	35.0	168.0	43.1	2

In [15…

```
from sklearn.impute import SimpleImputer

X = med.drop(columns="disease"); y = med["disease"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

imp = SimpleImputer(strategy="median")            # medians learned on th
sc  = StandardScaler()
X_train_s = sc.fit_transform(imp.fit_transform(X_train))
X_test_s  = sc.transform(imp.transform(X_test))

from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(max_iter=1000).fit(X_train_s, y_train)
y_pred = clf.predict(X_test_s)
y_prob = clf.predict_proba(X_test_s)[:, 1]

print("Confusion matrix:\n", metrics.confusion_matrix(y_test, y_pred))
print(f"\nAccuracy: {metrics.accuracy_score(y_test, y_pred):.3f} | AUC: {
print(metrics.classification_report(y_test, y_pred, target_names=["Health

imp_coef = pd.Series(clf.coef_[0], index=X.columns).sort_values()
plt.figure(figsize=(6.5, 4))
imp_coef.plot.barh(color=["tab:red" if v > 0 else "tab:blue" for v in imp
plt.title("Standardised coefficients (effect on log-odds of disease)"); p
```

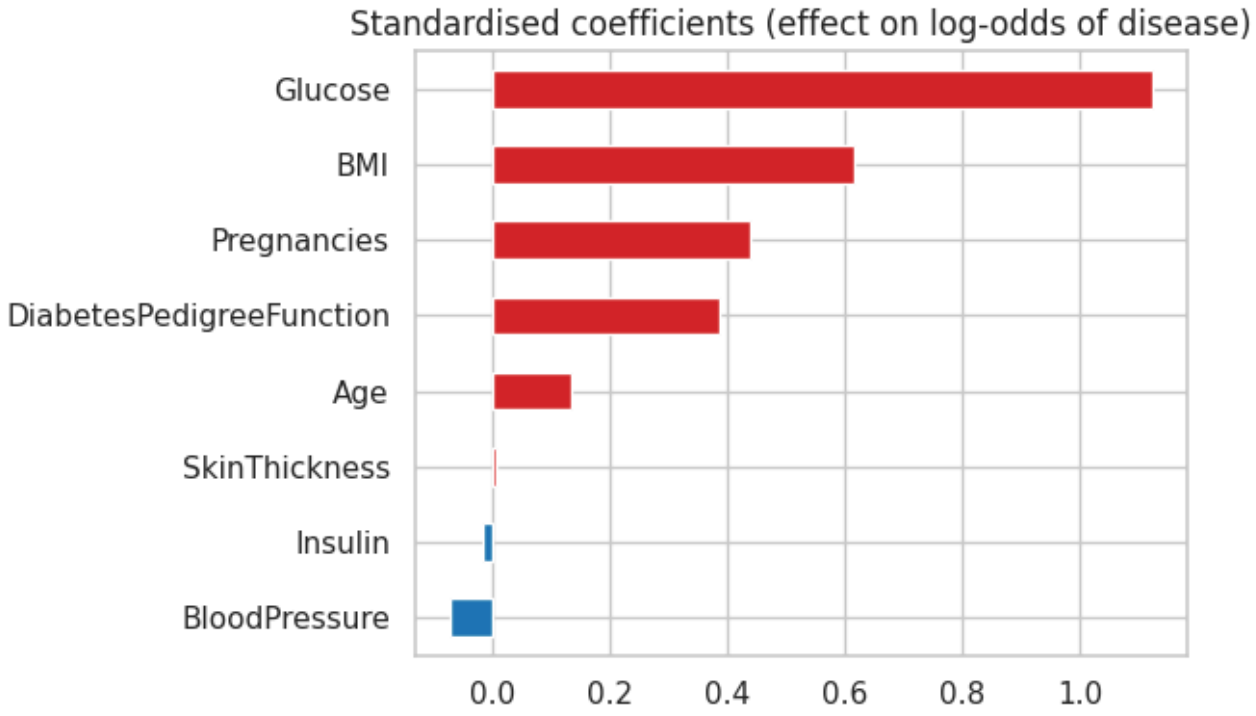
```
Confusion matrix:
 [[91  9]
 [28 26]]

Accuracy: 0.760 | AUC: 0.830

              precision    recall  f1-score   support

     Healthy       0.76      0.91      0.83       100
 Susceptible       0.74      0.48      0.58        54

    accuracy                           0.76       154
   macro avg       0.75      0.70      0.71       154
weighted avg       0.76      0.76      0.74       154
```



In [16…

```python
# Risk prediction for one new patient
patient = pd.DataFrame([{"Pregnancies": 2, "Glucose": 150, "BloodPressure
                         "Insulin": 160, "BMI": 33.0, "DiabetesPedigreeFu
p = clf.predict_proba(sc.transform(imp.transform(patient)))[0, 1]
print(f"Probability of being susceptible: {p:.1%}  ->  {'Susceptible' if
```

```
Probability of being susceptible: 56.3%  ->  Susceptible
```

Exercise 3 – Which product is a customer most likely to buy? (online retailer)

The exercise describes 3 product classes (**multinomial logistic regression**, softmax); the real dataset below has **2 outcomes** (buys / does not buy), so

the same multinomial code runs with two classes and would work unchanged with more.

> **Dataset – Social Network Ads (Kaggle: `rakeshrau/social-network-ads`).** Link: https://www.kaggle.com/datasets/rakeshrau/social-network-ads **Records:** 400 users · **Features:** `Age`, `EstimatedSalary`, `Gender` (one-hot) · **Target:** `product` (`Buys product` = `Purchased` 1 / `Does not buy` = 0). **Preprocessing:** dropped `User ID`; Gender one-hot encoded; features standardised; stratified 80/20 split. **Why it fits:** real customer data (age, income, purchase decision) for an online-advertising / purchase scenario; no real multi-class product column was available, so the target is binary.

In [17…
```
ads = load_data("Social_Network_Ads.csv",
                "https://raw.githubusercontent.com/sharmaroshan/Social-Ne
cust = ads.drop(columns="User ID")
cust["product"] = cust.pop("Purchased").map({1: "Buys product", 0: "Does

print(cust.shape, "| nulls:", cust.isnull().sum().sum())
print(cust["product"].value_counts(), "\n")
cust.head()
```

```
(400, 4) | nulls: 0
product
Does not buy    257
Buys product    143
Name: count, dtype: int64
```

Out[17]:

	Gender	Age	EstimatedSalary	product
0	Male	19.0	19000.0	Does not buy
1	Male	35.0	20000.0	Does not buy
2	Female	26.0	43000.0	Does not buy
3	Female	27.0	57000.0	Does not buy
4	Male	19.0	76000.0	Does not buy

In [18…
```
X = pd.get_dummies(cust.drop(columns="product"), columns=["Gender"], drop
y = cust["product"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

sc = StandardScaler()
X_train_s, X_test_s = sc.fit_transform(X_train), sc.transform(X_test)

from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(max_iter=1000).fit(X_train_s, y_train)      # mu
```

```
y_pred = clf.predict(X_test_s)

print(f"Accuracy: {metrics.accuracy_score(y_test, y_pred):.3f}\n")
print(metrics.classification_report(y_test, y_pred))

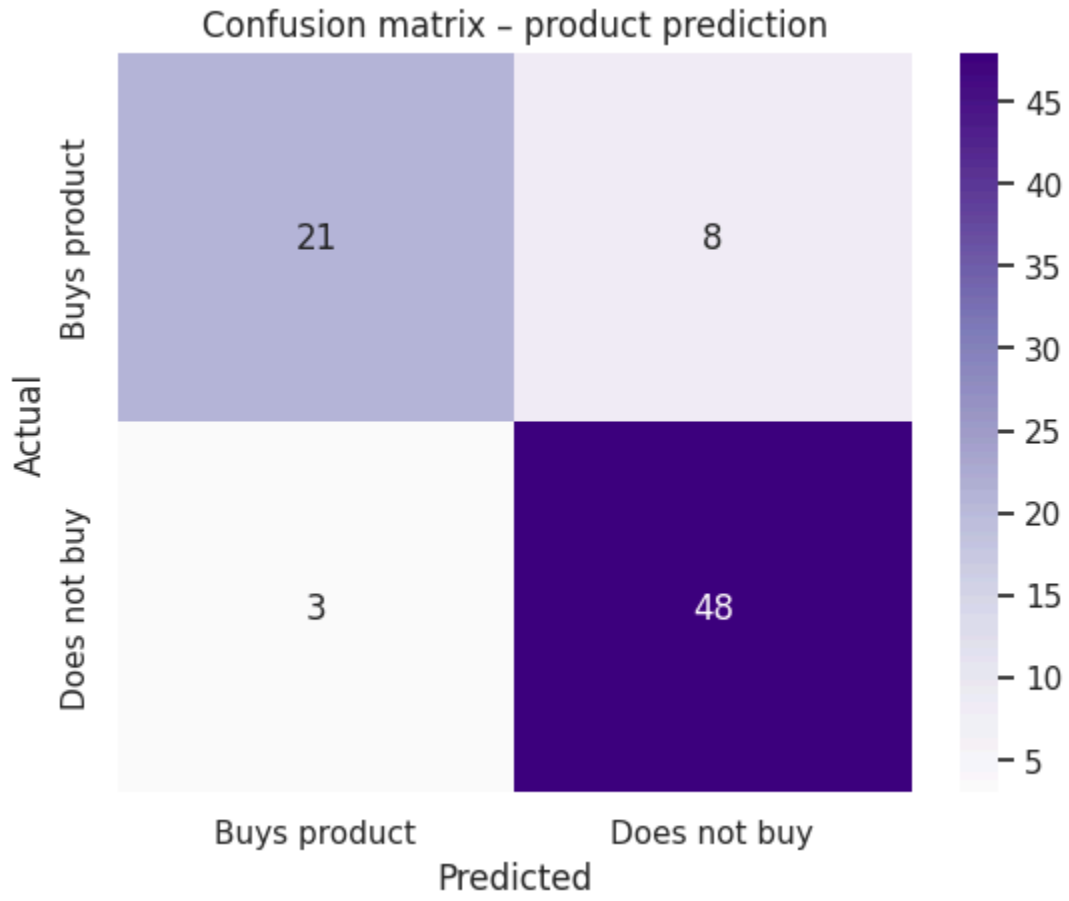
labels = clf.classes_
sns.heatmap(pd.DataFrame(metrics.confusion_matrix(y_test, y_pred, labels=
            annot=True, fmt="d", cmap="Purples")
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title("Confusion matri
```

```
Accuracy: 0.863

              precision    recall  f1-score   support

Buys product       0.88      0.72      0.79        29
Does not buy       0.86      0.94      0.90        51

    accuracy                           0.86        80
   macro avg       0.87      0.83      0.84        80
weighted avg       0.86      0.86      0.86        80
```



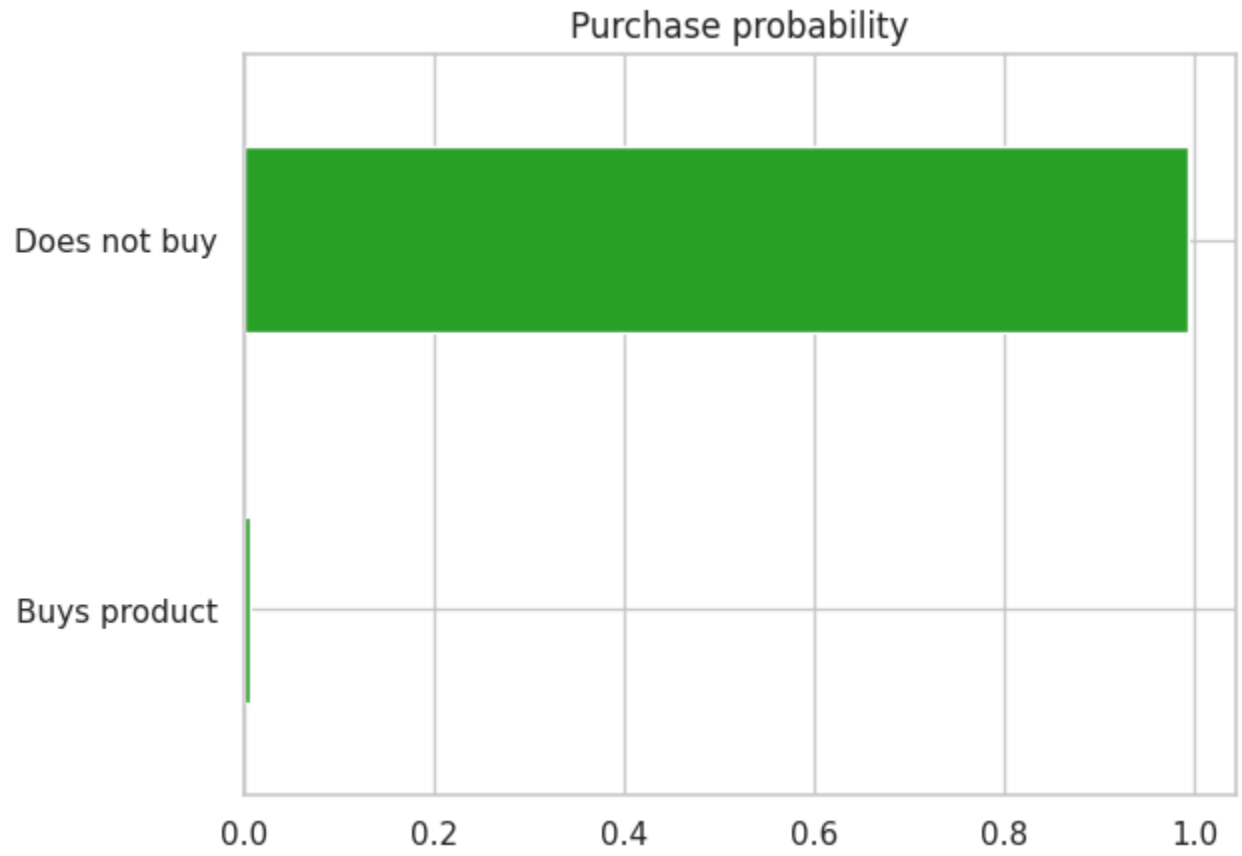
In [19…

```
# Will this customer buy?  (24 years old, estimated salary 45,000, female
new_customer = pd.DataFrame([{"Age": 24, "EstimatedSalary": 45000, "Gende
proba = pd.Series(clf.predict_proba(sc.transform(new_customer))[0], index
print(proba.round(3).to_string())
```

```
print(f"\nMost likely outcome -> {proba.idxmax()}  ({proba.max():.1%})")
proba.sort_values().plot.barh(color="tab:green", title="Purchase probabil
```

```
Buys product    0.006
Does not buy    0.994

Most likely outcome -> Does not buy  (99.4%)
```



7.6 Decision Tree Classifier – Exercises

Exercise 1 – Will someone buy a house? (age & income)

Scenario from the book: a tree splits on **age** and **income** to separate buyers from non-buyers, choosing the split that gains the most purity (entropy / Gini).

> **Dataset – California Housing Prices (Kaggle: `camnugent/california-housing-prices`, file `housing.csv`).** Link:

> https://www.kaggle.com/datasets/camnugent/california-housing-prices **Records:** 20,640 California census block groups · **Features:** `age` (= `housing_median_age`, years), `income` (= `median_income` × $10,000) · **Target:** `bought_house` = 1 if `median_house_value` is above the median of all districts (≈ $179,700), else 0. **Preprocessing:** two columns selected and renamed, target created by thresholding a real column; no nulls in these columns. `housing_median_age` is capped at 52 and `median_house_value` at $500,001 in the original data. **Caveat:** each row is a *district*, so `age` is the age of the housing stock and `income` the district's median income, not one person's age/income – a stand-in for the exercise's individual-level table.

In [20…
```
cal = load_data("housing.csv",
                "https://raw.githubusercontent.com/ageron/handson-ml/mast
house = pd.DataFrame({
    "age":    cal["housing_median_age"],
    "income": (cal["median_income"] * 10_000).round(-2),
})
house["bought_house"] = (cal["median_house_value"] > cal["median_house_va

print(house.shape, "| nulls:", house.isnull().sum().sum())
print("\nShare above the median house value:")
print(house.groupby(house.age > 30)["bought_house"].mean().rename({True:
house.head()
```

```
(20640, 3) | nulls: 0

Share above the median house value:
age
age <= 30    0.482
age > 30     0.521
Name: bought_house, dtype: float64
```

Out[20]:

	age	income	bought_house
0	41.0	83300.0	1
1	21.0	83000.0	1
2	52.0	72600.0	1
3	52.0	56400.0	1
4	52.0	38500.0	1

In [21…
```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text

X = house[["age", "income"]]; y = house["bought_house"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

```
tree = DecisionTreeClassifier(criterion="entropy", max_depth=2, random_st
y_pred = tree.predict(X_test)

print("Confusion matrix:\n", metrics.confusion_matrix(y_test, y_pred))
print(f"Accuracy: {metrics.accuracy_score(y_test, y_pred):.3f}\n")
print(export_text(tree, feature_names=["age", "income"]))
print("Feature importance:\n" + pd.Series(tree.feature_importances_, inde
```

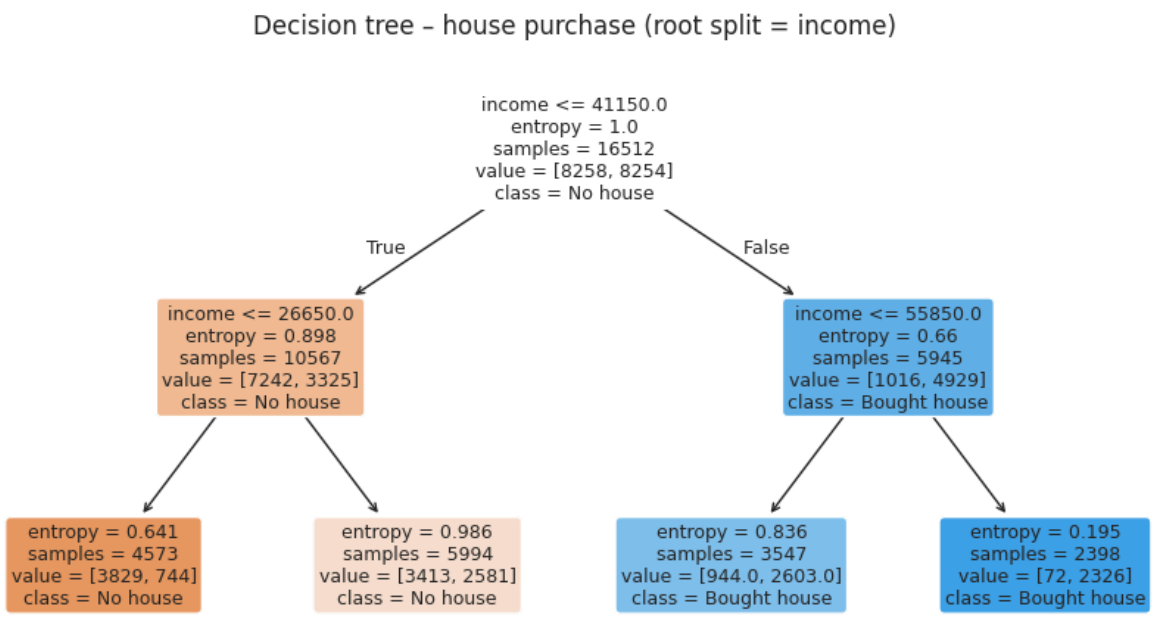
```
Confusion matrix:
 [[1798  267]
 [ 775 1288]]
Accuracy: 0.748

|--- income <= 41150.00
|   |--- income <= 26650.00
|   |   |--- class: 0
|   |--- income >  26650.00
|   |   |--- class: 0
|--- income >  41150.00
|   |--- income <= 55850.00
|   |   |--- class: 1
|   |--- income >  55850.00
|   |   |--- class: 1

Feature importance:
age       0.0
income    1.0
```

In [22…

```
root_feature = ["age", "income"][tree.tree_.feature[0]]
plt.figure(figsize=(11, 5.5))
plot_tree(tree, feature_names=["age", "income"], class_names=["No house",
          filled=True, rounded=True, fontsize=9)
plt.title(f"Decision tree — house purchase (root split = {root_feature})"
```



In [23…

```python
# Purity gained by the first (root) split (Gini impurity), as described i
def gini(y): p = y.mean(); return 2 * p * (1 - p)
root_feature = X_train.columns[tree.tree_.feature[0]]; threshold = tree.t
left, right = y_train[X_train[root_feature] <= threshold], y_train[X_trai
w = len(left) / len(y_train)
print(f"Root split                 : {root_feature} <= {threshold:,.1f}")
print(f"Impurity before split      : {gini(y_train):.3f}")
print(f"Impurity after split       : {w*gini(left) + (1-w)*gini(right):.3
print(f"'Bought' share, {root_feature} <= {threshold:,.1f} : {left.mean()
older = y_train[X_train.age > 30]
print(f"Over-30 (age > 30) 'bought' share : {older.mean():.0%}")
```

```
Root split                 : income <= 41,150.0
Impurity before split      : 0.500
Impurity after split       : 0.378
'Bought' share, income <= 41,150.0 : 31%   |   income > 41,150.0 : 83%
Over-30 (age > 30) 'bought' share : 52%
```

Exercise 2 – Go out or stay in? (weather)

Sunny → picnic / drink / errands (**Go out**); rain → **Stay in**. Categorical weather features are one-hot encoded.

> **Dataset – Play Tennis (Kaggle: `fredericobreno/play-tennis`, file `play_tennis.csv`).** Link: https://www.kaggle.com/datasets/fredericobreno/play-tennis **Records:** 14 days · **Features:** `Outlook`, `Temperature`, `Humidity`, `Wind` · **Target:** `Decision` (`play` = Yes → **Go out**, No → **Stay in**).

> **Preprocessing:** columns renamed and the Yes/No label re-worded to the exercise's Go out / Stay in; one-hot encoding of the four categorical features. **Why it fits:** the classic weather-based go-out-or-not table used to teach decision trees; small enough to read the whole tree.

In [24…
```
tennis = load_data("play_tennis.csv",
                   "https://raw.githubusercontent.com/jenny-hub90/Decisio
weather = tennis.rename(columns={"outlook": "Outlook", "temp": "Temperatu
                                 "wind": "Wind", "play": "Decision"})
weather["Decision"] = weather["Decision"].map({"Yes": "Go out", "No": "St
print(weather.shape)
weather
```

```
(14, 5)
```

Out[24]:

	Outlook	Temperature	Humidity	Wind	Decision
0	Sunny	Hot	High	Weak	Stay in
1	Sunny	Hot	High	Strong	Stay in
2	Overcast	Hot	High	Weak	Go out
3	Rain	Mild	High	Weak	Go out
4	Rain	Cool	Normal	Weak	Go out
5	Rain	Cool	Normal	Strong	Stay in
6	Overcast	Cool	Normal	Strong	Go out
7	Sunny	Mild	High	Weak	Stay in
8	Sunny	Cool	Normal	Weak	Go out
9	Rain	Mild	Normal	Weak	Go out
10	Sunny	Mild	Normal	Strong	Go out
11	Overcast	Mild	High	Strong	Go out
12	Overcast	Hot	Normal	Weak	Go out
13	Rain	Mild	High	Strong	Stay in

In [25…
```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text

X = pd.get_dummies(weather.drop(columns="Decision")).astype(int)
y = weather["Decision"]

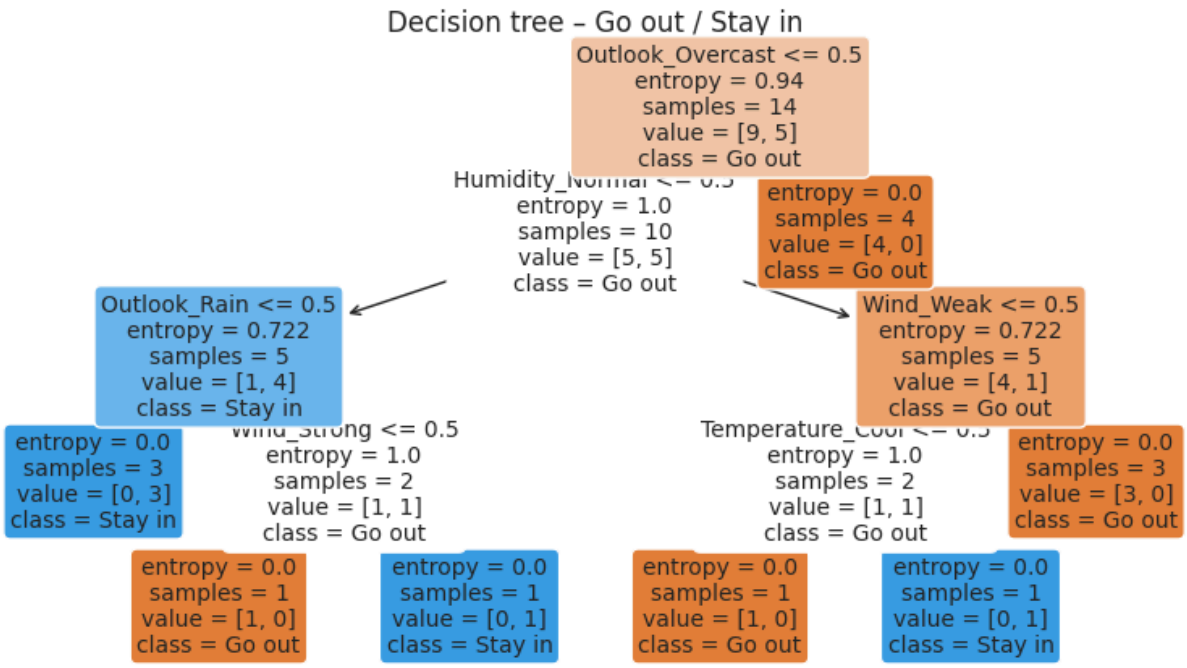
tree = DecisionTreeClassifier(criterion="entropy", random_state=0).fit(X,
print(f"Training accuracy: {tree.score(X, y):.2f}\n")
print(export_text(tree, feature_names=list(X.columns)))

plt.figure(figsize=(10, 5))
plot_tree(tree, feature_names=list(X.columns), class_names=list(tree.clas
```

```
plt.title("Decision tree — Go out / Stay in"); plt.show()
```

```
Training accuracy: 1.00

|--- Outlook_Overcast <= 0.50
|   |--- Humidity_Normal <= 0.50
|   |   |--- Outlook_Rain <= 0.50
|   |   |   |--- class: Stay in
|   |   |--- Outlook_Rain >  0.50
|   |   |   |--- Wind_Strong <= 0.50
|   |   |   |   |--- class: Go out
|   |   |   |--- Wind_Strong >  0.50
|   |   |   |   |--- class: Stay in
|   |--- Humidity_Normal >  0.50
|   |   |--- Wind_Weak <= 0.50
|   |   |   |--- Temperature_Cool <= 0.50
|   |   |   |   |--- class: Go out
|   |   |   |--- Temperature_Cool >  0.50
|   |   |   |   |--- class: Stay in
|   |   |--- Wind_Weak >  0.50
|   |   |   |--- class: Go out
|--- Outlook_Overcast >  0.50
|   |--- class: Go out
```



In [26…

```
# Predict for new days
new_days = pd.DataFrame([
    {"Outlook": "Sunny",    "Temperature": "Mild", "Humidity": "Normal",
    {"Outlook": "Rain",     "Temperature": "Hot",  "Humidity": "High",
    {"Outlook": "Overcast", "Temperature": "Cool", "Humidity": "High",
])
Xn = pd.get_dummies(new_days).reindex(columns=X.columns, fill_value=0).as
new_days["Prediction"] = tree.predict(Xn)
```

```
new_days
```

Out[26]:

	Outlook	Temperature	Humidity	Wind	Prediction
0	Sunny	Mild	Normal	Weak	Go out
1	Rain	Hot	High	Weak	Go out
2	Overcast	Cool	High	Strong	Go out

8.6 Support Vector Machines – Exercises

Exercise 1 – Random dataset following a quadratic distribution

Points are generated in 2-D; the class boundary is the parabola $x_2 = x_1^2$ (label 1 = above the parabola, 0 = below). A **linear** SVM cannot separate this, but **polynomial (degree 2)** and **RBF** kernels can.

> **Dataset note:** this is the one exercise whose text itself asks for a *generated* dataset ("random dataset following a quadratic distribution"), so the points are generated here on purpose (seed 0, 500 points) – there is no real dataset that has a known parabolic boundary. Exercise 2 below uses real data.

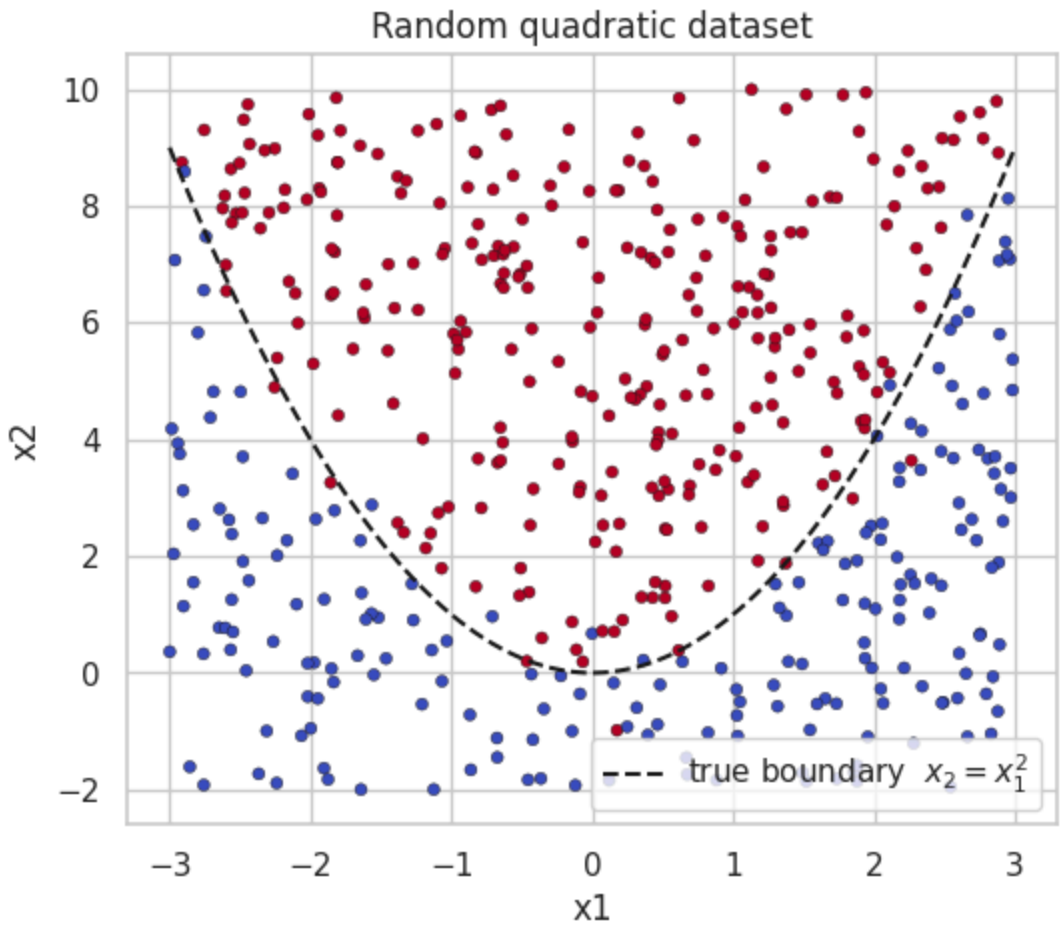
In [27…

```
rng = np.random.default_rng(0)
n = 500
x1 = rng.uniform(-3, 3, n)
x2 = rng.uniform(-2, 10, n)
label = (x2 > x1**2 + rng.normal(0, 0.5, n)).astype(int)     # quadratic

quad = pd.DataFrame({"x1": x1, "x2": x2, "class": label})
print(quad.shape, "| class balance:", quad["class"].value_counts().to_dic

plt.figure(figsize=(6, 5))
plt.scatter(x1, x2, c=label, cmap="coolwarm", s=18, edgecolor="k", linewi
xs = np.linspace(-3, 3, 200); plt.plot(xs, xs**2, "k--", label="true boun
plt.xlabel("x1"); plt.ylabel("x2"); plt.title("Random quadratic dataset")
```

```
(500, 3) | class balance: {1: 293, 0: 207}
```



In [28…

```
from sklearn.svm import SVC

X = quad[["x1", "x2"]].values; y = quad["class"].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,

models = {
    "Linear":              SVC(kernel="linear", C=1),
    "Polynomial (deg 2)": SVC(kernel="poly", degree=2, C=10, coef0=1, gam
    "RBF":                 SVC(kernel="rbf", C=10, gamma="scale"),
}
rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    rows.append({"Kernel": name, "Train acc": m.score(X_train, y_train),
                 "Test acc": m.score(X_test, y_test), "Support vectors":
pd.DataFrame(rows).round(3)
```

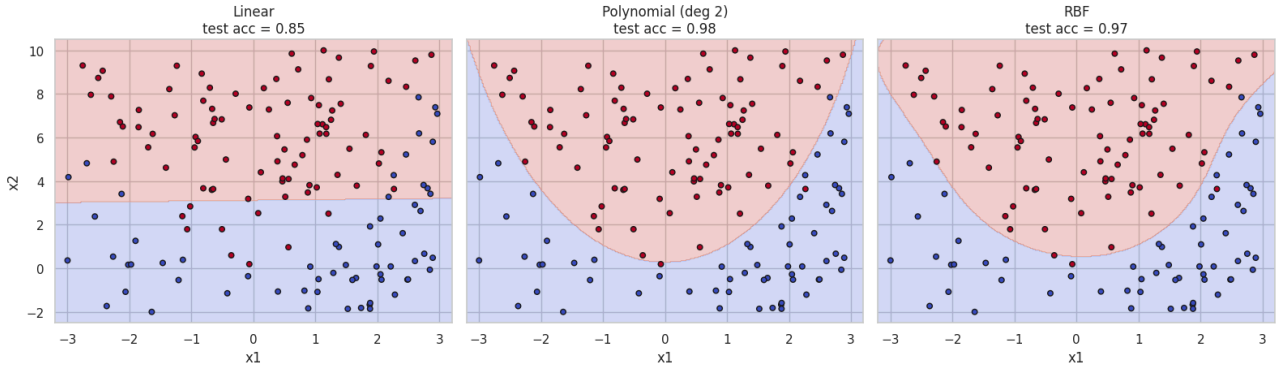
Out[28]:

	Kernel	Train acc	Test acc	Support vectors
0	Linear	0.803	0.847	163
1	Polynomial (deg 2)	0.969	0.980	77
2	RBF	0.966	0.973	76

In [29…

```
# Decision regions for the three kernels
xx, yy = np.meshgrid(np.linspace(-3.2, 3.2, 300), np.linspace(-2.5, 10.5,
fig, axes = plt.subplots(1, 3, figsize=(16, 4.8), sharey=True)
for ax, (name, m) in zip(axes, models.items()):
    Z = m.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.25, cmap="coolwarm")
    ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap="coolwarm", edg
    ax.set_title(f"{name}\ntest acc = {m.score(X_test, y_test):.2f}"); ax
axes[0].set_ylabel("x2"); plt.tight_layout(); plt.show()

print(metrics.classification_report(y_test, models["RBF"].predict(X_test)
```




```
                precision    recall  f1-score   support

Below parabola       0.95      0.98      0.97        62
Above parabola       0.99      0.97      0.98        88

      accuracy                           0.97       150
     macro avg       0.97      0.97      0.97       150
  weighted avg       0.97      0.97      0.97       150
```

Exercise 2 – Classify messages as spam / not spam (word frequency + message length)

> **Dataset – SMS Spam Collection (Kaggle: `uciml/sms-spam-collection-dataset`, UCI SMS Spam Collection).** Link: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset **Records:** 5,572 real SMS messages (5,169 after removing 403 exact duplicates; 653 spam / 4,516 ham) · **Target:** `spam` (1 = spam). **Features (built from the message text):** word-frequency percentages – `freq_free` ("free"), `freq_win` ("win/won/winner/prize"), `freq_call` ("call/txt/mobile/claim") and the ham-typical `freq_lol` ("lol/ok/sure/later") – plus `message_length`, `num_links`, `num_digits` and `capital_ratio`. **Preprocessing:** duplicates dropped (so

> identical messages cannot appear in both train and test); features standardised inside the pipeline; hyper-parameters chosen with 5-fold grid search. **Why it fits:** genuine spam / ham messages, and the exercise asks for word-frequency and message-length features.

In [30…

```
sms = load_data("sms.tsv", "https://raw.githubusercontent.com/justmarkham
                sep="\t", header=None, names=["label", "text"])
n_raw = len(sms)
sms = sms.drop_duplicates().reset_index(drop=True)
print(f"{n_raw} messages -> {len(sms)} after dropping exact duplicates")

def sms_features(text):
    """Word-frequency percentages + simple length / character features fo
    words = re.findall(r"[a-z']+", text.lower())
    n = max(len(words), 1)
    pct = lambda vocab: round(100 * sum(w in vocab for w in words) / n, 2
    letters = [c for c in text if c.isalpha()]
    return {
        "freq_free":      pct({"free"}),
        "freq_win":       pct({"win", "won", "winner", "prize"}),
        "freq_call":      pct({"call", "txt", "mobile", "claim"}),
        "freq_lol":       pct({"lol", "ok", "sure", "later"}),          #
        "message_length": len(text),
        "num_links":      len(re.findall(r"www\.|http", text.lower())),
        "num_digits":     sum(c.isdigit() for c in text),
        "capital_ratio":  round(sum(c.isupper() for c in letters) / max(l
    }

mail = pd.DataFrame([sms_features(t) for t in sms["text"]])
mail["spam"] = (sms["label"] == "spam").astype(int)
print(mail.shape, "| nulls:", mail.isnull().sum().sum(), "| spam share:",
mail.head()
```

```
5572 messages -> 5169 after dropping exact duplicates
(5169, 9) | nulls: 0 | spam share: 0.126
```

Out[30]:

	freq_free	freq_win	freq_call	freq_lol	message_length	num_links	num_digits	capital_ra
0	0.00	0.00	0.00	0.00	111	0	0	0.0
1	0.00	0.00	0.00	16.67	29	0	0	0.1
2	3.45	3.45	3.45	0.00	155	0	25	0.1
3	0.00	0.00	0.00	0.00	49	0	0	0.0
4	0.00	0.00	0.00	0.00	61	0	0	0.0

In [31…

```
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
```

```
X = mail.drop(columns="spam"); y = mail["spam"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,

pipe = Pipeline([("scale", StandardScaler()), ("svc", SVC(kernel="rbf"))]
grid = GridSearchCV(pipe, {"svc__C": [0.1, 1, 10, 100], "svc__gamma": ["s
grid.fit(X_train, y_train)

print("Best parameters :", grid.best_params_)
print(f"Best CV F1      : {grid.best_score_:.3f}\n")

y_pred = grid.predict(X_test)
print("Confusion matrix:\n", metrics.confusion_matrix(y_test, y_pred))
print(f"\nTest accuracy: {metrics.accuracy_score(y_test, y_pred):.3f}\n")
print(metrics.classification_report(y_test, y_pred, target_names=["Not sp

sns.heatmap(pd.crosstab(y_test, y_pred, rownames=["Actual"], colnames=["P
plt.title("SVC [rbf] — spam filter"); plt.show()
```

```
Best parameters : {'svc__C': 10, 'svc__gamma': 'scale'}
Best CV F1      : 0.926

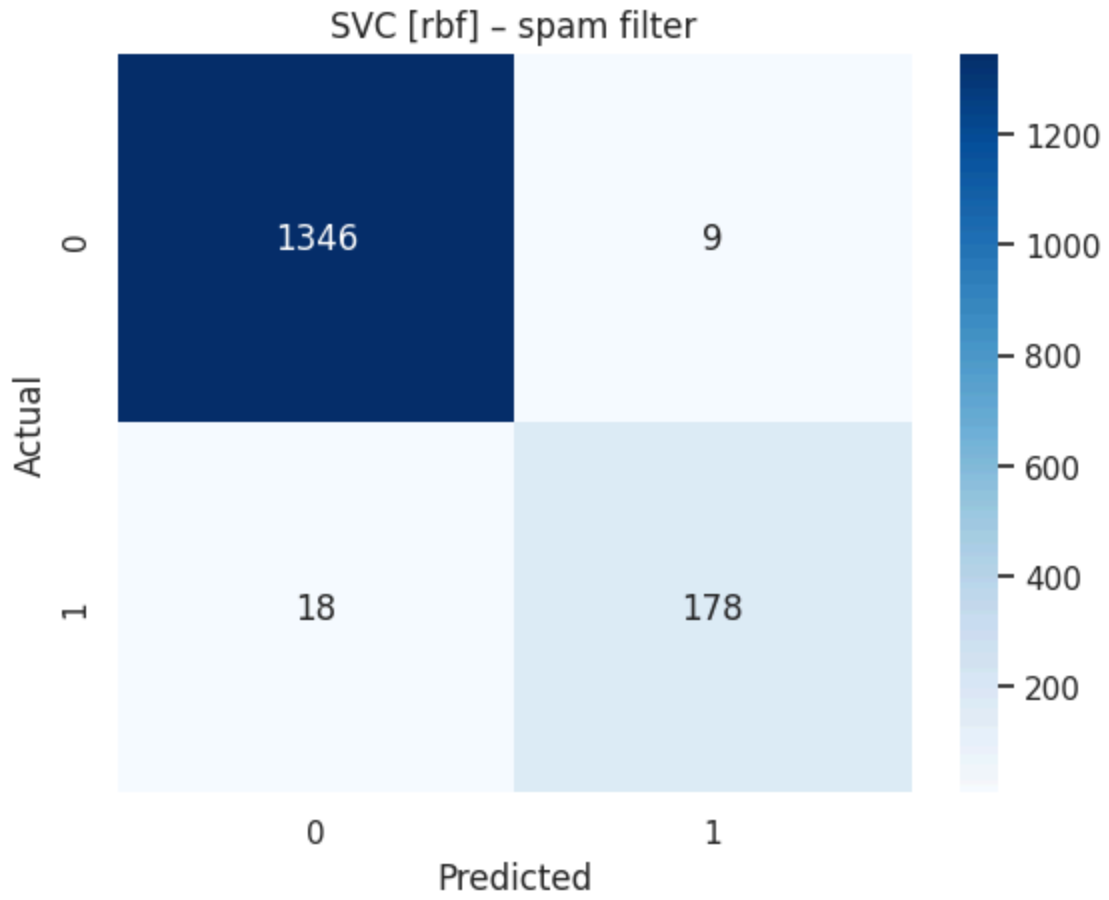
Confusion matrix:
 [[1346    9]
 [  18  178]]

Test accuracy: 0.983

              precision    recall  f1-score   support

    Not spam       0.99      0.99      0.99      1355
        Spam       0.95      0.91      0.93       196

    accuracy                           0.98      1551
   macro avg       0.97      0.95      0.96      1551
weighted avg       0.98      0.98      0.98      1551
```



In [32…

```
# Classify two new messages
new_msgs = [
    "WINNER!! You have won a FREE prize. Call 09061701461 now to claim yo
    "Ok lol, I'll be there later. Sure, see you at the meeting tomorrow",
]
new_mail = pd.DataFrame([sms_features(t) for t in new_msgs])
new_mail["prediction"] = np.where(grid.predict(new_mail[X.columns]) == 1,
new_mail.insert(0, "message", [m[:45] + "..." for m in new_msgs])
new_mail
```

Out[32]:

	message	freq_free	freq_win	freq_call	freq_lol	message_length	num_links	num_digits
0	WINNER!! You have won a FREE prize. Call 0906...	5.88	23.53	17.65	0.00	98	1	11
1	Ok lol, I'll be there later. Sure, see you at...	0.00	0.00	0.00	30.77	66	0	0

9.6 K-Nearest Neighbours – Exercises

Exercise 1 – Recognise handwritten digits with KNN

Uses scikit-learn's built-in **Digits** dataset (1 797 images, 8 × 8 pixels, classes 0-9).

In [33…

```
from sklearn.datasets import load_digits
digits = load_digits()
X, y = digits.data, digits.target
print("Images:", X.shape, "| classes:", np.unique(y))

fig, axes = plt.subplots(2, 8, figsize=(12, 3.4))
for ax, img, lab in zip(axes.ravel(), digits.images, digits.target):
    ax.imshow(img, cmap="gray_r"); ax.set_title(f"label {lab}", fontsize=
plt.suptitle("Sample handwritten digits"); plt.show()
```

```
Images: (1797, 64) | classes: [0 1 2 3 4 5 6 7 8 9]
```



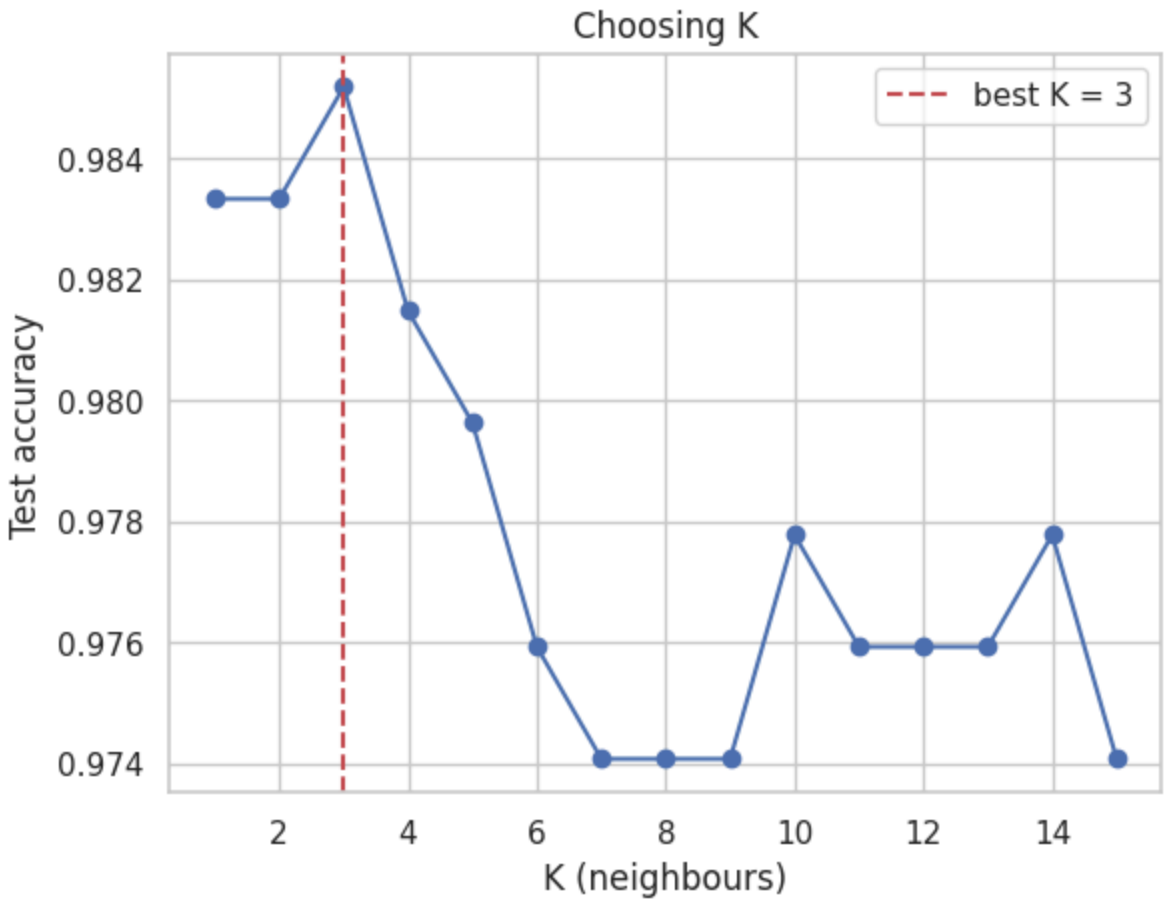
In [34…

```
from sklearn.neighbors import KNeighborsClassifier

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,

# Choose K by comparing accuracy for K = 1..15
ks = range(1, 16)
acc = [KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train).score(X_
best_k = ks[int(np.argmax(acc))]

plt.plot(ks, acc, "o-"); plt.axvline(best_k, color="r", ls="--", label=f"
plt.xlabel("K (neighbours)"); plt.ylabel("Test accuracy"); plt.title("Cho
print("Accuracy per K:", [round(a, 4) for a in acc])
```




```
Accuracy per K: [0.9833, 0.9833, 0.9852, 0.9815, 0.9796, 0.9759, 0.9741,
0.9741, 0.9741, 0.9778, 0.9759, 0.9759, 0.9759, 0.9778, 0.9741]
```

In [35…

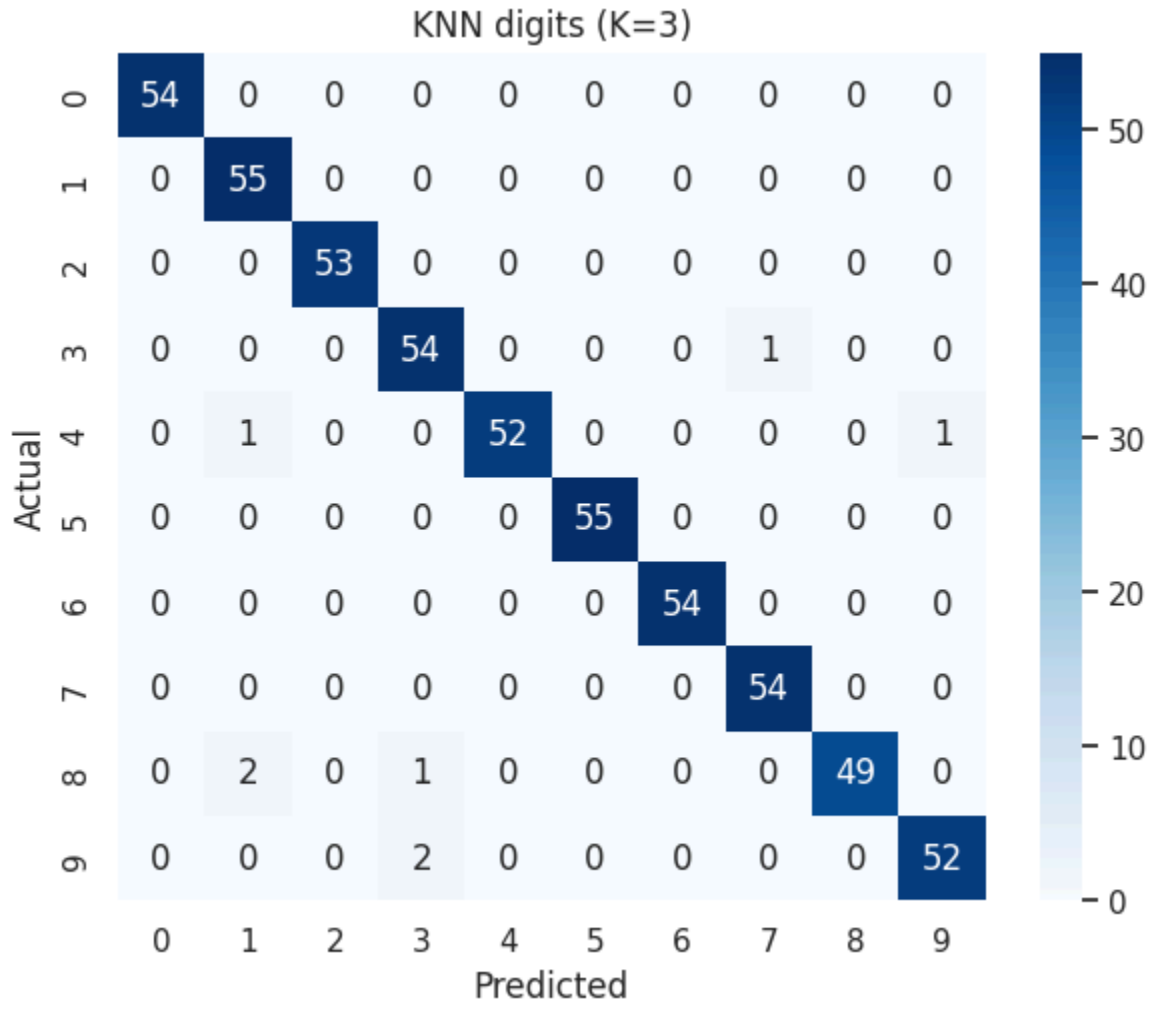
```
knn = KNeighborsClassifier(n_neighbors=best_k, metric="minkowski", p=2).f
y_pred = knn.predict(X_test)

print(f"KNN (K={best_k}) accuracy: {metrics.accuracy_score(y_test, y_pred
print(metrics.classification_report(y_test, y_pred))

plt.figure(figsize=(7, 5.5))
sns.heatmap(metrics.confusion_matrix(y_test, y_pred), annot=True, fmt="d"
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title(f"KNN digits (K=
```

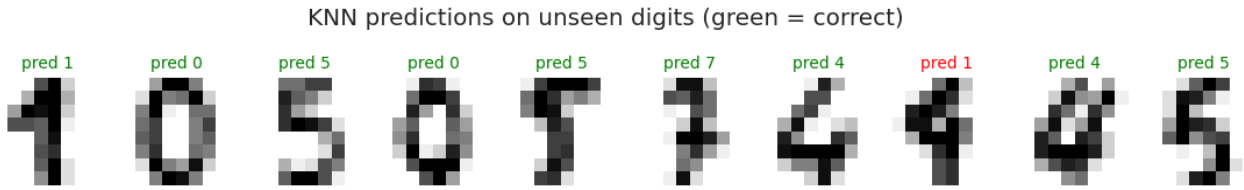
```
KNN (K=3) accuracy: 0.9852
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	54
1	0.95	1.00	0.97	55
2	1.00	1.00	1.00	53
3	0.95	0.98	0.96	55
4	1.00	0.96	0.98	54
5	1.00	1.00	1.00	55
6	1.00	1.00	1.00	54
7	0.98	1.00	0.99	54
8	1.00	0.94	0.97	52
9	0.98	0.96	0.97	54
accuracy			0.99	540
macro avg	0.99	0.99	0.99	540
weighted avg	0.99	0.99	0.99	540



In [36…
```
# Identify some new digits by appearance similarity to the samples
idx = np.random.default_rng(1).choice(len(X_test), 10, replace=False)
```

```
fig, axes = plt.subplots(1, 10, figsize=(14, 2.2))
for ax, i in zip(axes, idx):
    ax.imshow(X_test[i].reshape(8, 8), cmap="gray_r"); ax.axis("off")
    pred = knn.predict(X_test[i:i+1])[0]
    ax.set_title(f"pred {pred}", color="green" if pred == y_test[i] else
plt.suptitle("KNN predictions on unseen digits (green = correct)"); plt.s
```



Exercise 2 – Euclidean distance between a new entry and the existing data

Distance formula: $d = \sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$. The new entry is not specified in the table, so the classic example **Brightness = 20, Saturation = 35** is used (change `new_entry` to try others).

In [37…
```
colours = pd.DataFrame({
    "Brightness": [40, 50, 60, 10, 70, 60, 25],
    "Saturation": [20, 50, 90, 25, 70, 10, 80],
    "Class":      ["Red", "Blue", "Blue", "Red", "Blue", "Red", "Blue"],
})
new_entry = (20, 35)          # (Brightness, Saturation)

colours["Distance"] = np.sqrt((colours["Brightness"] - new_entry[0])**2 +
ranked = colours.sort_values("Distance").reset_index(drop=True)
ranked.index += 1
ranked.rename_axis("Rank").round(2)
```

Out[37]:

Rank	Brightness	Saturation	Class	Distance
1	10	25	Red	14.14
2	40	20	Red	25.00
3	50	50	Blue	33.54
4	25	80	Blue	45.28
5	60	10	Red	47.17
6	70	70	Blue	61.03
7	60	90	Blue	68.01

In [38…
```
K = 5
top_k = ranked.head(K)
```

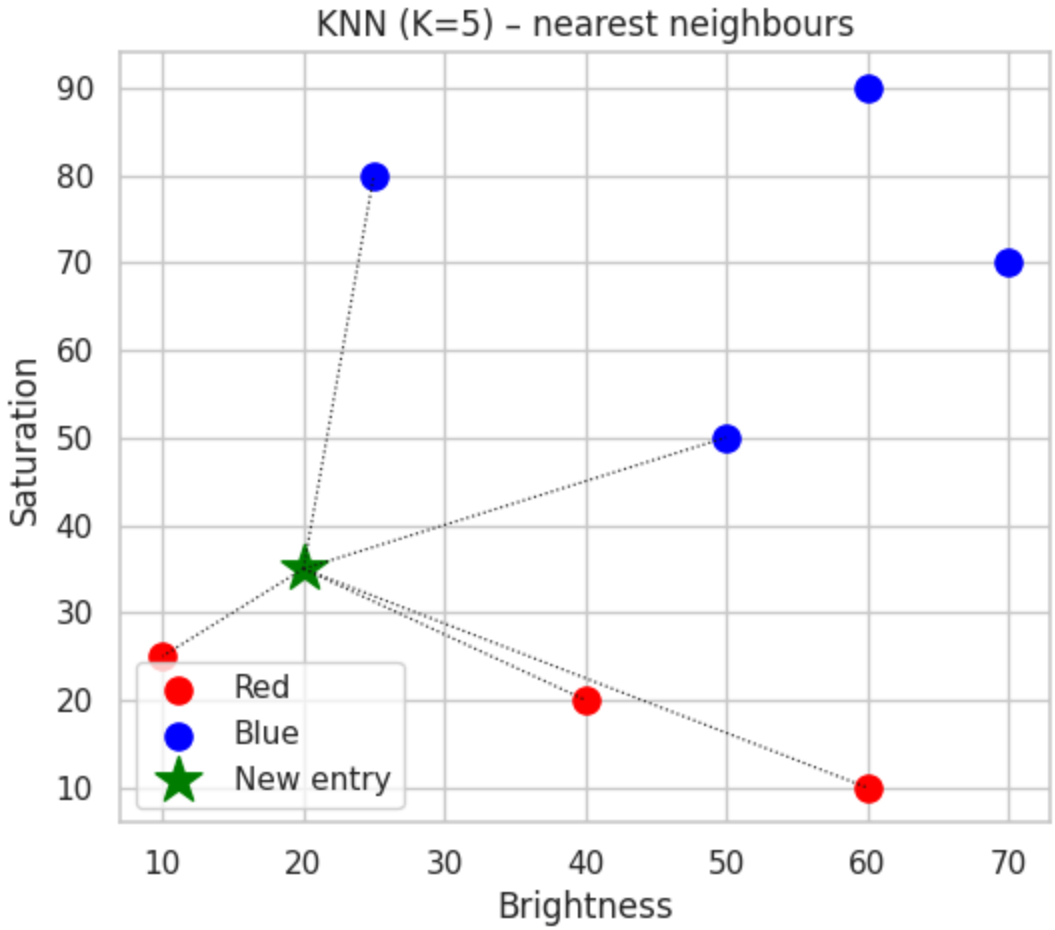
```
votes = top_k["Class"].value_counts()
print(f"{K} nearest neighbours of {new_entry}:")
print(top_k[["Brightness", "Saturation", "Class", "Distance"]].round(2).t
print("\nVotes:", votes.to_dict())
print(f"==> Predicted class for the new entry: {votes.idxmax()}")

# Cross-check with scikit-learn
from sklearn.neighbors import KNeighborsClassifier
sk = KNeighborsClassifier(n_neighbors=K).fit(colours[["Brightness", "Satu
print("scikit-learn KNN prediction        :", sk.predict([new_entry])[0])

plt.figure(figsize=(6, 5))
for c, col in [("Red", "red"), ("Blue", "blue")]:
    s = colours[colours.Class == c]; plt.scatter(s.Brightness, s.Saturati
plt.scatter(*new_entry, c="green", marker="*", s=300, label="New entry")
for _, r in top_k.iterrows():
    plt.plot([new_entry[0], r.Brightness], [new_entry[1], r.Saturation],
plt.xlabel("Brightness"); plt.ylabel("Saturation"); plt.title(f"KNN (K={K
```

```
5 nearest neighbours of (20, 35):
   Brightness  Saturation Class  Distance
1          10          25   Red     14.14
2          40          20   Red     25.00
3          50          50  Blue     33.54
4          25          80  Blue     45.28
5          60          10   Red     47.17

Votes: {'Red': 3, 'Blue': 2}
==> Predicted class for the new entry: Red
scikit-learn KNN prediction        : Red
```



10.6 K-Means Clustering – Exercises

Exercise 1 – Old Faithful geyser eruption segmentation

Cluster eruptions by **waiting time** between eruptions and **duration** of the eruption.

> **Dataset – Old Faithful geyser eruptions (the classic R `faithful` data; no Kaggle listing found).** Source used: https://raw.githubusercontent.com/gchoi/Dataset/master/OldFaithful.csv **Records:** 272 eruptions · **Features:** `duration` (= `TimeEruption`, min) and `waiting` (= `TimeWaiting`, min) · **Target:** none (unsupervised). **Preprocessing:** columns

> renamed; both features standardised before K-Means because their ranges differ a lot. **Why it fits:** it is the textbook two-cluster example (short eruption / short wait vs. long eruption / long wait).

In [39…
```
geyser_raw = load_data("OldFaithful.csv",
                       "https://raw.githubusercontent.com/gchoi/Dataset/m
geyser = geyser_raw.rename(columns={"TimeEruption": "duration", "TimeWait

print(geyser.shape, "| nulls:", geyser.isnull().sum().sum())
geyser.describe().round(2)
```

```
(272, 2) | nulls: 0
```

Out[39]:

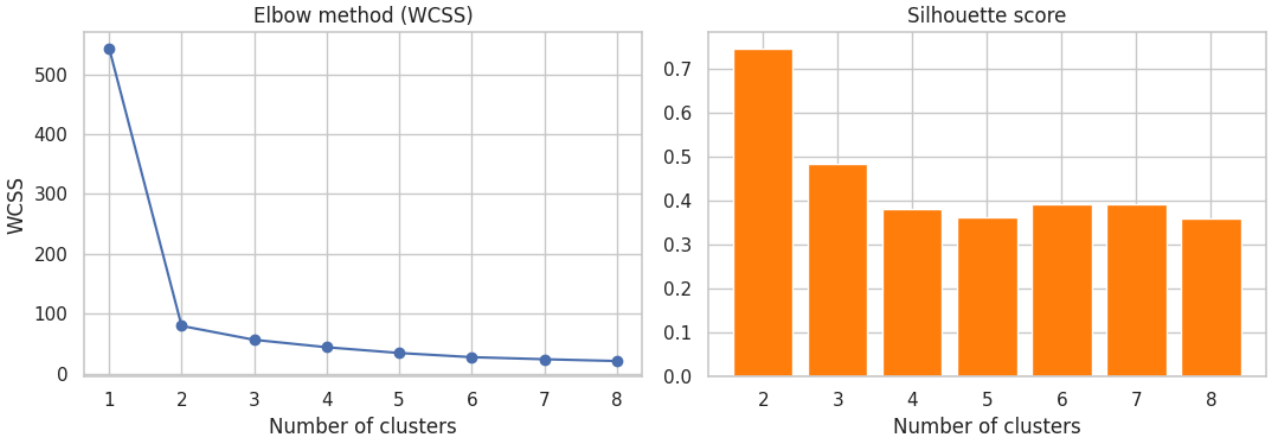
	duration	waiting
count	272.00	272.00
mean	3.49	70.90
std	1.14	13.59
min	1.60	43.00
25%	2.16	58.00
50%	4.00	76.00
75%	4.45	82.00
max	5.10	96.00

In [40…
```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

sc = StandardScaler()
Xg = sc.fit_transform(geyser[["waiting", "duration"]])       # scale: the

wcss, sil = [], {}
for k in range(1, 9):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=4
    wcss.append(km.inertia_)
    if k > 1: sil[k] = silhouette_score(Xg, km.labels_)

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(range(1, 9), wcss, "o-"); ax[0].set(title="Elbow method (WCSS)
ax[1].bar(list(sil.keys()), list(sil.values()), color="tab:orange"); ax[1
plt.tight_layout(); plt.show()
print({k: round(v, 3) for k, v in sil.items()})
```




```
{2: np.float64(0.745), 3: np.float64(0.485), 4: np.float64(0.382), 5: n
p.float64(0.363), 6: np.float64(0.392), 7: np.float64(0.392), 8: np.float
64(0.358)}
```

In [41…

```
model = KMeans(n_clusters=2, init="k-means++", n_init=10, random_state=42
geyser["cluster"] = model.fit_predict(Xg)
centers = pd.DataFrame(sc.inverse_transform(model.cluster_centers_), colu
centers.index.name = "cluster"

print("Cluster centres (original units):"); print(centers.to_string(), "\
print("Cluster sizes:"); print(geyser["cluster"].value_counts().sort_inde
print(f"Silhouette score: {silhouette_score(Xg, geyser['cluster']):.3f}")

plt.figure(figsize=(7, 5.5))
for c, col in zip([0, 1], ["tab:blue", "tab:orange"]):
    s = geyser[geyser.cluster == c]; plt.scatter(s.waiting, s.duration, c
plt.scatter(centers.waiting, centers.duration, c="black", s=220, marker="
plt.xlabel("Waiting time between eruptions (min)"); plt.ylabel("Eruption
plt.title("K-Means segmentation — Old Faithful geyser"); plt.legend(); pl
```

```
Cluster centres (original units):
         waiting  duration
cluster
0          80.08      4.30
1          54.59      2.05

Cluster sizes:
cluster
0    174
1     98

Silhouette score: 0.745
```



Exercise 2 – Image compression with K-Means (396 × 396 × 3)

Every pixel is an RGB point; K-Means reduces the image to **K representative colours** (each pixel is replaced by its cluster centre). A 396 × 396 × 3 crop of scikit-learn's bundled sample photo is used (swap in your own image with `PIL.Image.open(...).resize((396, 396))`).

In [42…

```
from sklearn.datasets import load_sample_image
china = load_sample_image("china.jpg")
img = china[10:406, 120:516]                        # crop to exactly 396
print("Image shape:", img.shape, "| dtype:", img.dtype)

plt.imshow(img); plt.axis("off"); plt.title("Original image (396 x 396 x
```

```
Image shape: (396, 396, 3) | dtype: uint8
```



In [43…

```
from sklearn.cluster import KMeans
import math

pixels = img.reshape(-1, 3) / 255.0                # 156,816 pixels x 3 c
h, w, _ = img.shape
results, compressed = [], {}

for k in [2, 4, 8, 16, 32]:
    km = KMeans(n_clusters=k, n_init=1, random_state=42).fit(pixels)
    recon = km.cluster_centers_[km.labels_].reshape(h, w, 3)
    compressed[k] = recon
    mse = np.mean((img / 255.0 - recon) ** 2)
    orig_bits = h * w * 24
    comp_bits = h * w * math.ceil(math.log2(k)) + k * 24        # index p
    results.append({"K colours": k, "bits/pixel": math.ceil(math.log2(k))
                    "PSNR (dB)": 10 * np.log10(1 / mse), "Compression rat

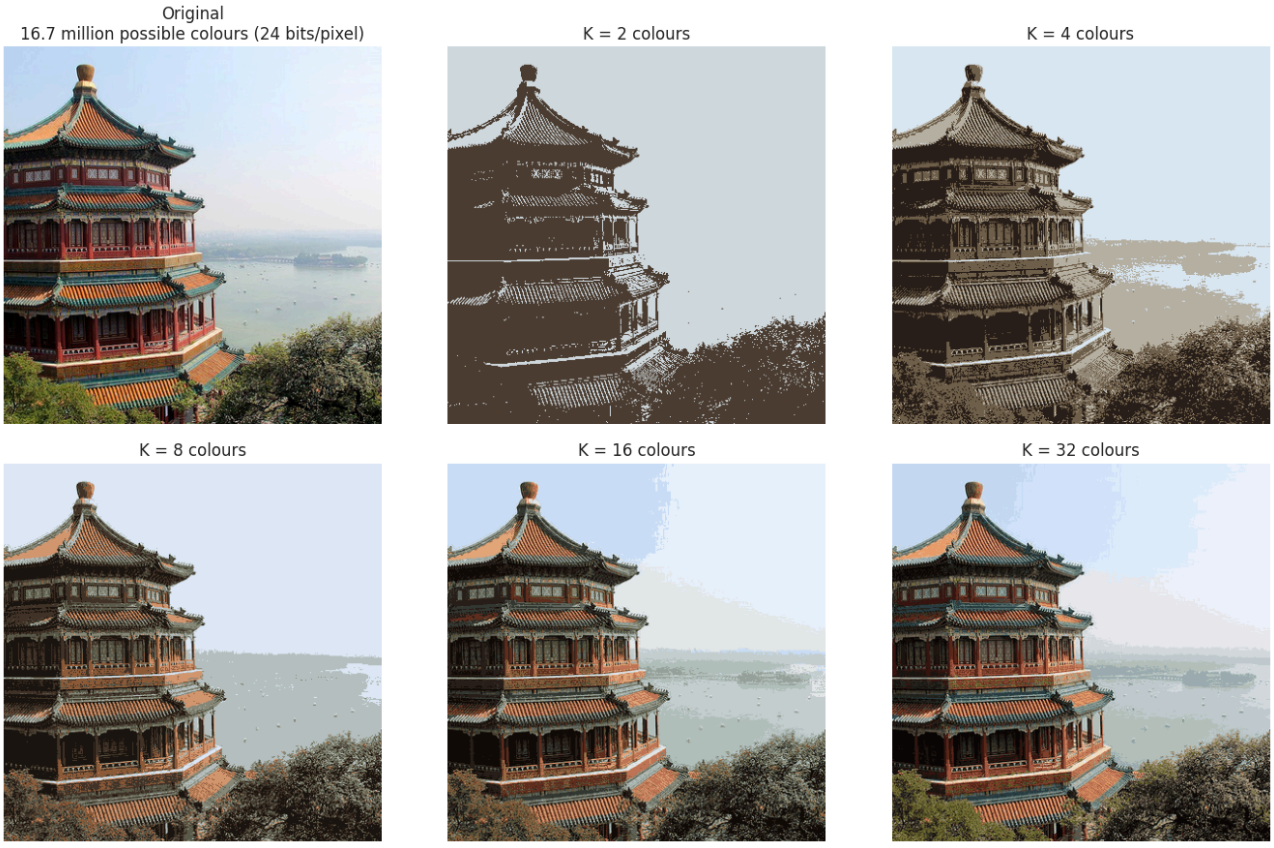
pd.DataFrame(results).round(2)
```

Out[43]:

	K colours	bits/pixel	PSNR (dB)	Compression ratio
0	2	1	17.10	23.99
1	4	2	21.47	12.00
2	8	3	24.49	8.00
3	16	4	27.21	6.00
4	32	5	29.79	4.80

In [44…

```
fig, axes = plt.subplots(2, 3, figsize=(14, 9))
axes[0, 0].imshow(img); axes[0, 0].set_title("Original\n16.7 million poss
for ax, (k, im) in zip(axes.ravel()[1:], compressed.items()):
    ax.imshow(im); ax.set_title(f"K = {k} colours")
for ax in axes.ravel(): ax.axis("off")
plt.tight_layout(); plt.show()
```



11.4 Principal Component Analysis – Exercises

Exercise 1 – Why are Maggie's properties not selling?

Approach: (1) compare two groups of properties, (2) use **PCA** to find the main dimensions along which the listings differ, (3) confirm the drivers with a model.

> **Dataset – House Sales in King County, USA (Kaggle: `harlfoster/housesalesprediction`, file `kc_house_data.csv`).** Link: https://www.kaggle.com/datasets/harlfoster/housesalesprediction **Records:** 21,613 real house sales (May 2014 – May 2015) · **Features (12):** `sqft_living`, `sqft_lot`, `bedrooms`, `bathrooms`, `floors`, `waterfront`, `view`, `condition`, `grade`, `age_years` (= 2015 − `yr_built`), `sqft_basement`, `sqft_living15` · **Outcome:** `high_price` = 1 if `price` is above the county median ($450,000). **Preprocessing:** `id`, `date`, `zipcode`, `lat`, `long`, `yr_renovated` and `sqft_lot15` dropped (location is therefore *not* part of the analysis); `age_years` derived; all features standardised before PCA. No missing values. **Caveat:** this dataset contains *sold* homes only – there is **no sold / unsold flag**. The closest real signal is the sale price, so the analysis asks *which property characteristics separate high-priced from lower-priced homes and how PCA summarises them*; re-run it on Maggie's own listings (with a sold/unsold column) to answer her exact question.

In [45…

```
kc = load_data("kc_house_data.csv",
               "https://raw.githubusercontent.com/Shreyas3108/house-price
props = pd.DataFrame({
    "price_k":       kc["price"] / 1000,                # used only to d
    "sqft_living":   kc["sqft_living"],
    "sqft_lot":      kc["sqft_lot"],
    "bedrooms":      kc["bedrooms"],
    "bathrooms":     kc["bathrooms"],
    "floors":        kc["floors"].astype(float),
    "waterfront":    kc["waterfront"],
    "view":          kc["view"],
    "condition":     kc["condition"],
```

```
    "grade":          kc["grade"],
    "age_years":      2015 - kc["yr_built"],
    "sqft_basement": kc["sqft_basement"],
    "sqft_living15": kc["sqft_living15"],
})
props["high_price"] = (props["price_k"] > props["price_k"].median()).asty

print(props.shape, "| nulls:", props.isnull().sum().sum())
print(f"High-priced (above the median of ${props.price_k.median():,.0f}k)
props.head()
```

```
(21613, 14) | nulls: 0
High-priced (above the median of $450k): 49.7%
```

Out[45]:

	price_k	sqft_living	sqft_lot	bedrooms	bathrooms	floors	waterfront	view	condition	g
0	221.9	1180	5650	3	1.00	1.0	0	0	3	
1	538.0	2570	7242	3	2.25	2.0	0	0	3	
2	180.0	770	10000	2	1.00	1.0	0	0	3	
3	604.0	1960	5000	4	3.00	1.0	0	0	5	
4	510.0	1680	8080	3	2.00	1.0	0	0	3	

In [46…

```
# Step 1 — compare the two groups of properties
comp = props.groupby("high_price").mean().T
comp.columns = ["Lower-priced", "High-priced"]
comp["High - Lower"] = comp["High-priced"] - comp["Lower-priced"]
comp.round(2)
```

Out[46]:

	Lower-priced	High-priced	High - Lower
price_k	318.97	763.57	444.60
sqft_living	1623.50	2541.18	917.68
sqft_lot	11627.44	18623.72	6996.27
bedrooms	3.13	3.61	0.49
bathrooms	1.81	2.42	0.61
floors	1.37	1.62	0.26
waterfront	0.00	0.01	0.01
view	0.07	0.40	0.33
condition	3.39	3.43	0.05
grade	7.05	8.27	1.22
age_years	44.95	43.02	-1.93
sqft_basement	187.60	396.53	208.93
sqft_living15	1668.42	2308.09	639.68

In [47…
```
# Step 2 — PCA on standardised features
from sklearn.decomposition import PCA

feat = [c for c in props.columns if c not in ("price_k", "high_price")]
Xs = StandardScaler().fit_transform(props[feat])

pca = PCA().fit(Xs)
evr = pca.explained_variance_ratio_
print("Explained variance ratio :", evr.round(3))
print("Cumulative               :", evr.cumsum().round(3))

loadings = pd.DataFrame(pca.components_[:3].T, index=feat, columns=["PC1"
print("\nLoadings of the first 3 principal components:")
loadings.round(3)
```

```
Explained variance ratio : [0.349 0.158 0.106 0.084 0.066 0.056 0.047 0.0
45 0.037 0.021 0.02  0.01 ]
Cumulative               : [0.349 0.507 0.613 0.697 0.763 0.819 0.867 0.9
12 0.949 0.969 0.99  1.   ]

Loadings of the first 3 principal components:
```

Out[47]:

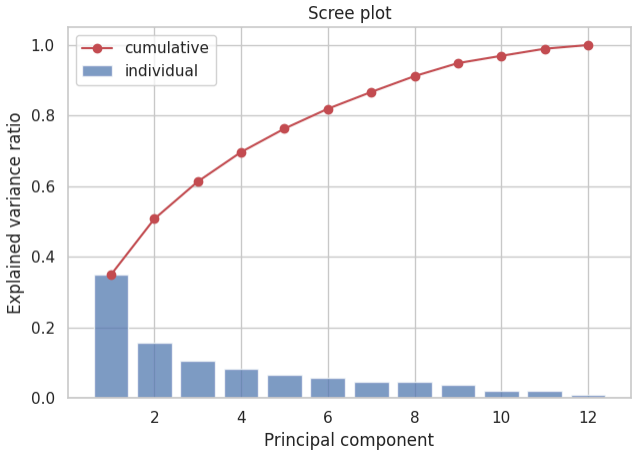
	PC1	PC2	PC3
sqft_living	0.445	0.149	-0.109
sqft_lot	0.080	0.054	0.077
bedrooms	0.289	0.152	-0.358
bathrooms	0.423	-0.037	-0.096
floors	0.265	-0.402	0.143
waterfront	0.068	0.233	0.673
view	0.159	0.360	0.522
condition	-0.094	0.408	-0.220
grade	0.419	-0.043	0.037
age_years	-0.267	0.422	-0.058
sqft_basement	0.152	0.504	-0.215
sqft_living15	0.392	0.070	0.003

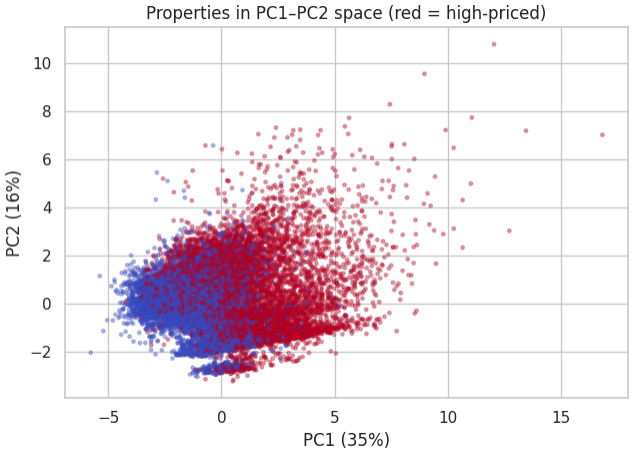
In [48…
```
scores = pca.transform(Xs)
fig, ax = plt.subplots(1, 2, figsize=(13, 4.8))
ax[0].bar(range(1, len(evr) + 1), evr, alpha=0.7, label="individual"); ax
ax[0].set(title="Scree plot", xlabel="Principal component", ylabel="Expla

ax[1].scatter(scores[:, 0], scores[:, 1], c=props["high_price"], cmap="co
ax[1].set(title="Properties in PC1–PC2 space (red = high-priced)", xlabel
plt.tight_layout(); plt.show()

# how strongly does each component separate the two groups? (mean score p
```

```
sep = pd.DataFrame(scores[:, :3], columns=["PC1", "PC2", "PC3"]).groupby(
sep.index = ["Lower-priced", "High-priced"]
print("Mean component score per group:"); print(sep.round(2).to_string())
```






```
Mean component score per group:
                PC1   PC2   PC3
Lower-priced -1.01 -0.23  0.02
High-priced   1.02  0.24 -0.02
```

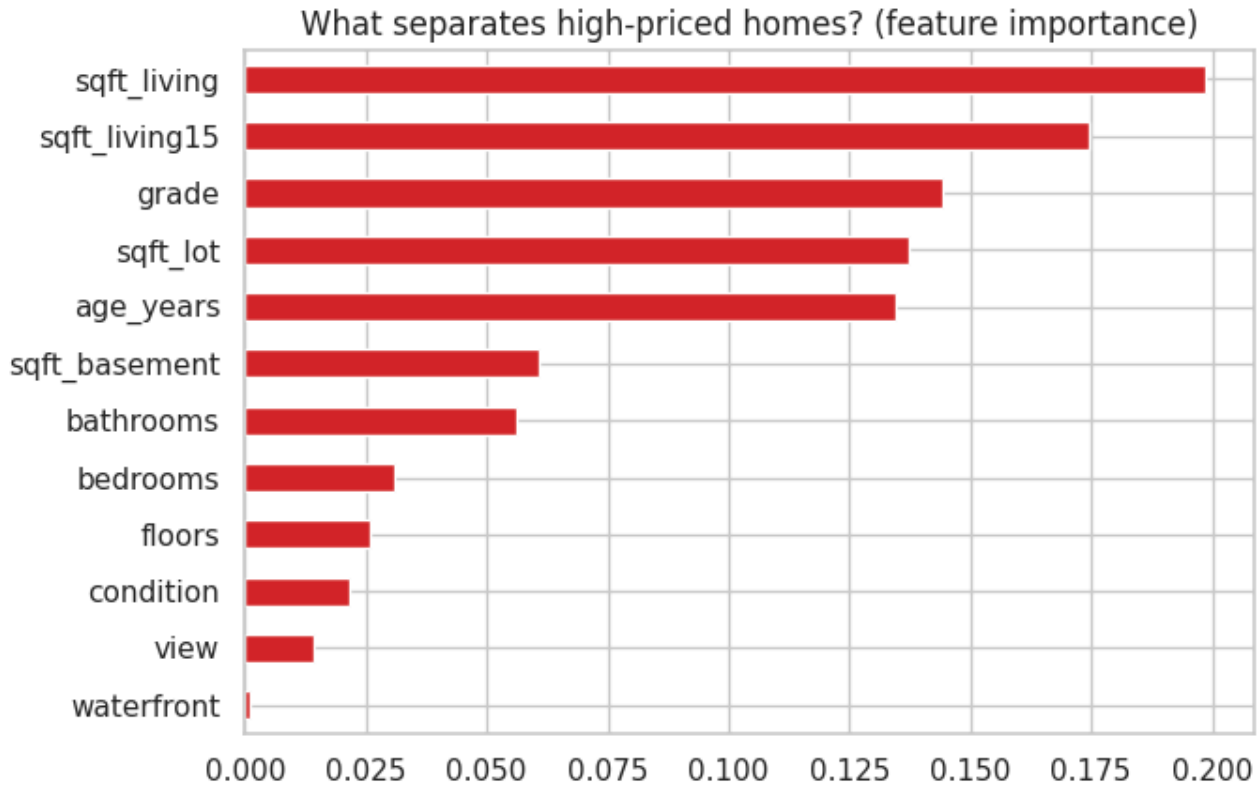
In [49…

```
# Step 3 — which features actually drive "high price"?  (Random Forest im
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rf = RandomForestClassifier(n_estimators=200, random_state=0, n_jobs=-1)
print(f"5-fold CV accuracy: {cross_val_score(rf, props[feat], props['high
rf.fit(props[feat], props["high_price"])

imp = pd.Series(rf.feature_importances_, index=feat).sort_values()
imp.plot.barh(color="tab:red", figsize=(7, 4.5), title="What separates hi
plt.tight_layout(); plt.show()
print(imp.sort_values(ascending=False).round(3).to_string())
```

```
5-fold CV accuracy: 0.817
```




```
sqft_living      0.199
sqft_living15    0.175
grade            0.144
sqft_lot         0.137
age_years        0.135
sqft_basement    0.061
bathrooms        0.056
bedrooms         0.031
floors           0.026
condition        0.022
view             0.014
waterfront       0.001
```

Insights for Maggie

Finding (from the tables/plots above)	What it means
Size and build quality separate high-priced from lower-priced homes – high-priced homes average ≈ 2,541 sq ft of living space vs ≈ 1,624 (+57 %), a construction `grade` of 8.3 vs 7.1, 2.4 vs 1.8 bathrooms and far more views (0.40 vs 0.07).	Buyers in the upper price band expect a large, well-built house; a listing that is small or low-grade for its asking price is hard to justify.
PCA structure – PC1 (≈ 35 % of variance) is a *size / quality* axis (`sqft_living` 0.45, `bathrooms` 0.42, `grade` 0.42, `sqft_living15` 0.39) and it is the only component that	The listings vary mainly in *how big and well built they are*; a single

Finding (from the tables/plots above)	What it means
separates the groups (mean score −1.01 vs +1.02). PC2 (≈ 16 %) contrasts *older, basement-heavy, higher-condition* homes with *multi-storey* ones, and PC3 (≈ 11 %) is *waterfront / view*; neither separates the groups on average. Eight components are needed to keep ≈ 91 % of the variance.	component (PC1) already summarises most of what makes a home “premium”.
Random-forest check – 5-fold CV accuracy ≈ 0.82; the most important features are `sqft_living` (≈ 0.20), `sqft_living15` (≈ 0.18) and `grade` (≈ 0.14). `condition` (≈ 0.02) and `waterfront` (≈ 0.00) matter little here.	Confirms the PCA reading: size and grade dominate. Waterfront homes are rare (< 1 % of sales), so their low importance reflects rarity, not low value.
Age and condition barely differ – average age ≈ 43 vs 45 years and condition score 3.43 vs 3.39.	Cosmetic condition or age alone is not what puts a home in a different price band in this data.

Recommendation: compare each stale listing's size and grade with the comparable sales *in the same price band*; if the house is smaller or lower-grade than those comparables, the asking price – not the marketing – is the likely obstacle, so re-price rather than spend on cosmetic work. *(Caveat: this dataset has no sold / unsold flag and no location variables in the analysis, so these are drivers of **price band**, used as a stand-in. Re-run the notebook on Maggie's own listings with a sold / unsold column to obtain her actual drivers.)*

Exercise 2 – Identify which variable is the principal component

The table gives the **loadings** (weights) of nine variables on the first three components. A variable is a *significant contributor* to a component when its absolute loading is large (rule of thumb: |loading| ≥ 0.5 – the values highlighted in bold in the exercise).

In [50…

```
load = pd.DataFrame({
    "PC1": [0.190, 0.544, 0.782, 0.365, 0.585, 0.394, 0.985, 0.520, 0.142
    "PC2": [0.017, 0.020, -0.605, 0.294, 0.085, -0.273, 0.126, 0.402, 0.1
    "PC3": [0.207, 0.204, 0.144, 0.585, 0.234, 0.027, -0.111, 0.519, 0.23
}, index=["Climate", "Housing", "Health", "Crime", "Transportation", "Edu
load.index.name = "Variable"

# Highlight |loading| >= 0.5 and find the dominant variable of each compo
print("Dominant variable in each component (largest |loading|):")
for pc in load.columns:
    v = load[pc].abs().idxmax()
```

```
    print(f"  {pc}: {v:<15} loading = {load.loc[v, pc]:+.3f}")

print("\nVariables with |loading| >= 0.5:")
for pc in load.columns:
    sig = load.index[load[pc].abs() >= 0.5].tolist()
    print(f"  {pc}: {sig}")

load.style.map(lambda v: "font-weight:bold;background-color:#fde68a" if a
```

```
Dominant variable in each component (largest |loading|):
  PC1: Arts            loading = +0.985
  PC2: Health          loading = -0.605
  PC3: Crime           loading = +0.585

Variables with |loading| >= 0.5:
  PC1: ['Housing', 'Health', 'Transportation', 'Arts', 'Recreation']
  PC2: ['Health']
  PC3: ['Crime', 'Recreation']
```

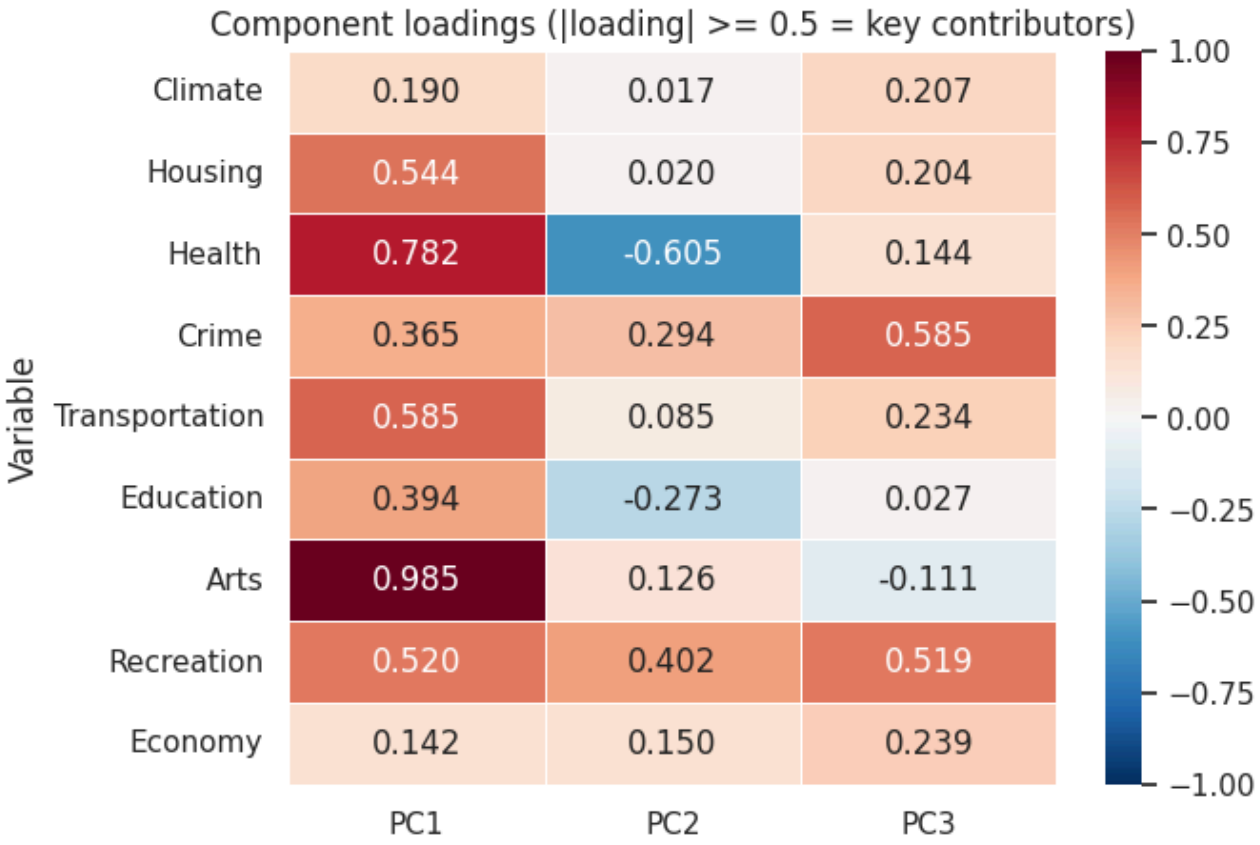
Out[50]:

Variable	PC1	PC2	PC3
Climate	0.190	0.017	0.207
Housing	**0.544**	0.020	0.204
Health	**0.782**	**-0.605**	0.144
Crime	0.365	0.294	**0.585**
Transportation	**0.585**	0.085	0.234
Education	0.394	-0.273	0.027
Arts	**0.985**	0.126	-0.111
Recreation	**0.520**	0.402	**0.519**
Economy	0.142	0.150	0.239

In [51…

```
plt.figure(figsize=(6.5, 5))
sns.heatmap(load, annot=True, fmt=".3f", cmap="RdBu_r", center=0, vmin=-1
plt.title("Component loadings (|loading| >= 0.5 = key contributors)"); pl

# Sum of squared loadings on PC1..PC3 — overall importance of each variab
print((load ** 2).sum(axis=1).sort_values(ascending=False).round(3).renam
```




```
Variable
Arts              0.998
Health            0.998
Recreation        0.701
Crime             0.562
Transportation    0.404
Housing           0.338
Education         0.230
Economy           0.100
Climate           0.079
```

Interpretation

- **PC1** – dominated by **Arts (0.985)**; Health (0.782), Transportation (0.585), Housing (0.544) and Recreation (0.520) also load strongly. PC1 is essentially a *cultural / urban-amenities* component, and **Arts is the variable that best represents the first principal component**.
- **PC2** – only **Health (−0.605)** passes the 0.5 cut-off; it is contrasted with Recreation (+0.402) and Crime (+0.294), so PC2 is a *health vs. recreation* contrast.
- **PC3** – **Crime (0.585)** and **Recreation (0.519)** are the key variables.
- **Climate** and **Economy** have small loadings on all three components (sum of squares ≈ 0.08 and 0.10) – these three components hardly represent

them, while **Arts** and **Health** are almost completely explained (≈ 0.998).

> A principal component is a *linear combination of all variables*, not one variable; the variables with the largest |loading| are the ones that "define" it.

12.6 Random Forest – Exercise

Exercise 1 – Fruit basket: which fruit is likely to be taken often?

`n` bootstrap samples are drawn from the basket and one decision tree is built per sample. In classification, the forest returns the **majority vote** of the trees:

- Tree-1 → **Class A (Apple)**
- Tree-2 → **Class A (Apple)**
- Tree-n → **Class B (Banana)**

In [52…

```
from collections import Counter

tree_outputs = {"Tree-1": "Apple (Class A)", "Tree-2": "Apple (Class A)",
votes = Counter(tree_outputs.values())

print("Votes from the individual trees:")
for tree_name, out in tree_outputs.items():
    print(f"  {tree_name:<7} -> {out}")

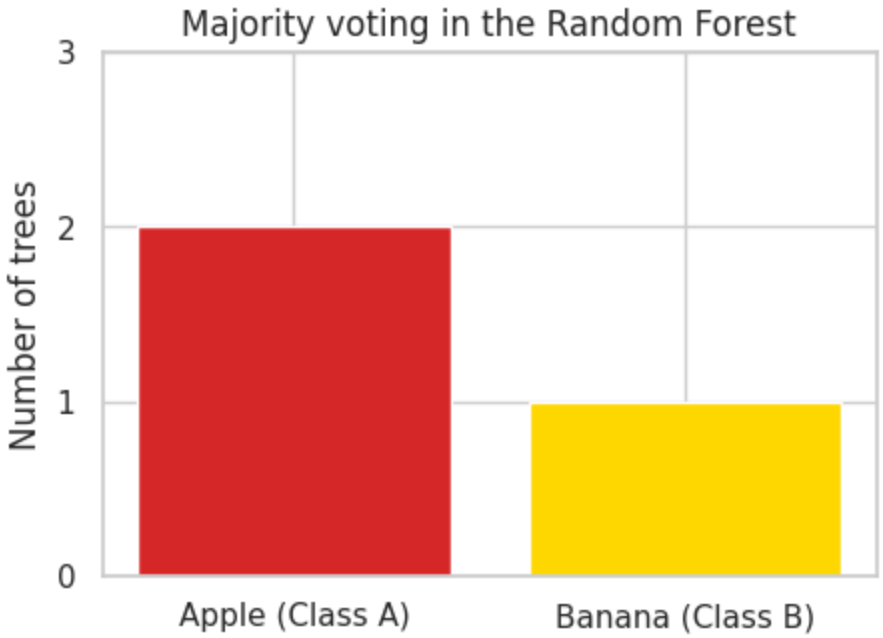
print("\nVote count:", dict(votes))
winner, count = votes.most_common(1)[0]
print(f"\nMajority voting -> Final class: {winner}  ({count} of {len(tree

plt.figure(figsize=(5, 3.4))
plt.bar(votes.keys(), votes.values(), color=["tab:red", "gold"])
plt.ylabel("Number of trees"); plt.title("Majority voting in the Random F
```

```
Votes from the individual trees:
  Tree-1  -> Apple (Class A)
  Tree-2  -> Apple (Class A)
  Tree-n  -> Banana (Class B)

Vote count: {'Apple (Class A)': 2, 'Banana (Class B)': 1}

Majority voting -> Final class: Apple (Class A)  (2 of 3 trees)
```



Answer: the **Apple** is the fruit likely to be taken most often – two of the three trees vote Class A, so the majority-voting step outputs **Apple** as the final class. (For regression the forest would instead *average* the trees' outputs.)